

Course Overview

Course Overview

My name is Mel Grubb, and welcome to my course, Object-oriented Design with C#. I'm a senior software developer from Columbus, Ohio. You've undoubtedly heard the phrase object-oriented programming in your career and have a pretty good idea of what it means in general. But it's a broad topic and the phrase can be kind of vague. In this course, we're going to learn about the principles and practices that make up object-oriented design and programming. Some of the major topics that we'll cover include the four pillars which define object orientation itself, as well as the SOLID principles which guide your code's design in a more relatable manner. From there, we'll cover design patterns that can give you an advantage when tackling common problems. We'll see how modern C# language features like explicit nullability and record types can be used to realize these designs. By the end of this course, you'll have a solid understanding of object-oriented programming concepts and how to implement them in C# 10. Before beginning this course, you should be familiar with the C# language in general and how to work with it in Visual Studio, including class libraries and command line projects. Entity Framework Core will be used in one of the demos, so having at least some familiarity will be helpful, but it isn't required. I hope you'll join me on this journey to learn these foundational development concepts with the Object-oriented Design with C# course at Pluralsight.

Introduction to Object-oriented Design

Introduction

Hi. I'm Mel Grubb, and welcome to my course, Object-oriented Design with C# 10. In this course, I'm going to cover a lot of ground, some of which you may already be familiar with. But I'll be talking about these subjects from a slightly different angle than you might be used to. I encourage you to watch the

parts of this course that you think you know already and maybe you'll start thinking about that topic in a new way or in a way that you've forgotten. For the demos in this course, I'll be using the following tools. The samples are presented in C# using Visual Studio 2022 with Mads Christensen's Markdown editor extension version 2 or higher. Unit tests will be written using NUnit 3 And the most current test adapter. In this course, we'll be looking at how C# language features can be used to implement object-oriented designs. Although this course is in C# the concepts apply to any object-oriented language, like Java, Python, Ruby, PHP, Visual Basic, and others. This course is for everyone and I don't want to leave anyone behind, so I'll try to keep everyone on the same page as we go along. Let's start off by clarifying our terminology. Object-oriented programming is not the same thing as object-oriented design. Writing code in an object-oriented language only makes your code object-oriented at a superficial level, but it does very little to influence the design of your system, and that distinction is important. You can absolutely write procedural code in C#. There's nothing stopping you from writing your whole program in the main method and using goto statements for flow control. Yes, the goto keyword still exists in C#, and it may even have the occasional legitimate use, but that's not the point. If you're going to write code like this, then why are you doing it in an object-oriented language to begin with? Conversely, there is nothing stopping you from writing object-oriented code in a procedural language. You'd be taking on a lot of the responsibilities that a proper object-oriented language gives you for free, but you could do it if you really wanted to for some reason. You could divide your code up into logical units and think of them as objects, each with variables to maintain their internal state and with global variables used to pass that state between those units. If you dig down deep enough under the covers down to the assembly or machine language that's really running things, that's what's actually happening anyway. Neither of these would be a good idea though. You should use your language to its strengths, and a proper object-oriented language makes implementing object-oriented designs easier. Not automatic, but easier. So what is object-oriented design then? In a way, it's just a commitment to thinking about the responsibilities of those classes and taking a disciplined approach to deciding how your code will be organized. Each class you write is an opportunity to create tools that the rest of the system can leverage. It's just object-oriented programming done right. That's all. So let's dive in and get started.

Demo: Mermaid

Before we continue, I want to introduce a tool that I'll be using throughout this course. One common way that developers illustrate and communicate their designs is through the use of diagrams. Class and sequence diagrams are just two of the standard set of diagram types known as the Unified Modeling Language, or UML. You've probably seen them many times whether you knew they were related or not. Many tools exist for drawing UML diagrams, but I'll be using Mermaid in this course because it's a simple way to create quick sketches of the ideas we'll be discussing. Mermaid is entirely Markdown based, and all you need to use it is a text editor that supports it. Mads Kristensen has created a free Markdown Editor extension for Visual Studio and it supports Mermaid, so that's what I'll be using. It means we'll be able to stay in Visual Studio the whole time we're discussing these concepts. In fact, I won't be writing any actual code at all in this module. We're just going to talk about the concepts and sketch components using Mermaid. I'm using Visual Studio 2022 for this course, so, I'll want to install Markdown Editor version 2 or higher. You can find it on the Visual Studio Marketplace site or from Visual Studio's Extension Manager. Just search for Markdown Editor, and make sure you choose the one labeled v2 or higher. Once the editor is installed and you've restarted Visual Studio, you'll need to change at least one setting. Head to Tools, Options, Text Editor, Markdown, Advanced, and disable Emoji IntelliSense. We'll be typing colons pretty often while creating class diagrams, and the editor will keep trying to suggest emojis if you don't disable this. Even with this disabled, it'll still try to double up the colons, though, but, it's slightly less irritating with this setting off. Now that the editor extension is installed, let's take a look at a simple Mermaid diagram. While we're working on a concept, it's often useful to start with something simple like a flow chart. I'll create a fresh Markdown document, start a code block, and specify Mermaid for the syntax type. So that's three backticks followed immediately by the word Mermaid. At this point, the diagram is showing an error because we haven't even told it what kind of diagram we want yet. For a flow chart, we use the keyword graph on a new line, followed by a two-letter code specifying the direction of flow. LR means left to right. Then you add lines describing the nodes in the graph. Describe the links between the nodes by drawing little ASCII arrows between them using two dashes and a greater than sign, like this. And now the preview window shows the completed graph. If I change that LR in the diagram to TD for top down, or TB for top to bottom, you'll see that the graph changes direction in the preview window. This is not a course on Mermaid, so, I'm not going to go into all of its details, but I'll be drawing diagrams using Mermaid as we go along, so, you'll get the idea. Mermaid is incredibly flexible and lets you describe your ideas by

following a few simple conventions. Here's a finished diagram with different shape nodes, line widths, and types, and with more descriptive names. You don't have any kind of fine control over how the nodes are positioned in a Mermaid diagram, but that's not the point. Concentrate on the information, not the layout. These diagrams aren't necessarily meant for publication, they're just an easy way to embed UML diagrams into Markdown documents. Flow charts are great for thinking through a problem, but, when we're working on designing our code, what we really need is a class diagram. Mermaid has us covered there as well. I'll start a new diagram, only this time, I'll start with the keyword, `classDiagram`. I'll define a class using the `class` keyword, and then add some properties to it. The preview window shows us the diagram, and it's probably something you've seen before. We can specify the types of the properties by adding them after the name, separated by a colon, and now the diagram includes the type information. Remember that UML is language agnostic, so this won't look like C# or even Visual Basic for that matter. There is even a convention for specifying visibility. Public members are prefixed with a plus sign, private members with a minus sign, protected members with a pound sign, and internal members get marked with a tilde. Unfortunately, tildes don't display properly in Mermaid at the moment because they're also being used in place of angle brackets for defining generic parameters, which you'll see in a little bit. I've submitted a bug report, and this issue may be fixed by the time you watch this, but, I'm going to use percent signs to indicate internal visibility for now. You might have been expecting some kind of visual change on the preview, but this really is the UML standard, you just need to remember what the symbols mean, or make yourself a cheat sheet. Additional conventions exist to specify whether a member is static or abstract. but for now, this is all we need to get started.

Pillars and Principles

The most important thing I want you to take away from this course is that object-oriented design means more than just the implementation details. As I said earlier, simply using an object-oriented language does not automatically make your code object-oriented. I'd like to take this further by stating that even using the object-oriented features of a language doesn't make your code, or more importantly, your design, object-oriented. You can use interfaces and inheritance, and still miss the bigger picture. Object-oriented design is about the larger concepts and principles at play, not about the language features used to implement them. Keep that in mind as we go through what might otherwise

seem like familiar topics. It's nearly impossible to have a conversation about object-oriented programming and not discuss two major topics, the four pillars and the SOLID principles. Let's start with the older of the two, the four pillars are our key features of object-oriented languages that make object-oriented programming and design possible. We'll explore the four pillars in depth in the next module, but to sum up, they are abstraction, you don't need to know how other classes work internally, it's none of your business. Encapsulation, you don't need to know what other classes know, if they want your help they'll ask for it. This is sometimes called data hiding. Inheritance, objects have a taxonomy, a family tree, if you will. A dog is a mammal, a lizard is a reptile, both are animals. And, polymorphism, a method that takes an animal will gladly accept a dog or a lizard. We've seen these examples countless times, but do you really stop to think about them when you're designing a system? If you're writing in an object-oriented language like C#, then they are ever present, and you may take them for granted. You should purposefully think about them when designing a system, though. The problem with the four pillars is that once you've boiled things down to just four words, you've lost an awful lot of information. It's up to you to remember exactly what those four words mean. There are other more descriptive and memorable guides to making good design decisions. One of the most famous of these comes from Robert Martin, better known in the industry as Uncle Bob. The SOLID principles are what he considered to be the five most important principles of object-oriented design. Single Responsibility Principle, a module should have a well-defined job. The Open-Closed Principle, you should provide extension points for customization. Liskov Substitution Principle, which is essentially polymorphism. Interface Segregation Principle, you should divide off multiple task-specific interfaces. And, Dependency Inversion Principle, which means don't tie yourself to specific implementations. Each of the five SOLID principles maps to one or more of the four pillars, and if you design systems with these principles in mind, they will gently steer you in the right direction to produce better, more maintainable code. In addition to the pillars and principles, there are several other key phrases that you'll commonly hear in conversations about object-oriented design and programming. You may hear additional terms, such as cohesion, coupling, and orthogonality. These are more abstract terms and they're all just different ways of describing the same core concepts, but, perhaps looking it from different angles. Let's go through some of them now. Cohesion describes how closely related the code is in a particular module. In this sense, the word module can refer to any grouping of code. This could mean a single method, or a class, a namespace, or even an entire project or solution.

Whatever scale you're looking at, cohesion refers to how well the elements at that level are focused on the single concept. Let's start with our simple person class from earlier. This class would be considered highly cohesive since every member of the class is concerned with representing a person. Suppose we wanted to format a person's name for display. We could put that code all over our application wherever a person's name appears and call it a front-end concern. Or, we could implement that code just once in the person class itself by giving it a full name property. That seems simple enough, and the person class is still highly cohesive since a person's full name is a property of that person. At a lower level, the full name property is internally cohesive since it has a single job, and that's all it does without any kind of side effects. No other code in our project needs to be concerned with how to format a person's name now, the person class will take care of it. If people are going to be added at runtime, we'll want to make sure all the required properties have been provided, so let's add a validate method to our person object. In the Mermaid diagram, I'll add a stub for the `IValidatableObject` interface with an annotation that it's really an interface and not a class. Mermaid doesn't distinguish between the classes and interfaces because neither does the UML spec, so, we'll use these annotations to clarify what kind of thing we're looking at on the diagram. Because angle brackets have special meaning in Markdown documents, Mermaid uses tildes instead to indicate generic parameters. You'll see that it still displays properly in the diagram. I'll add a dashed line and arrow from the person class to the `IValidatableObject` interface to show the relationship. Mermaid supports multiple different arrow styles. A closed arrow, like this one, is used to show inheritance or implementation. I can further clarify the relationship by adding a note that says implements, just to make it super obvious what's going on here. Occasionally, we'll want to send emails to our customers, perhaps to inform them of upcoming sales. We'll add a property to hold the person's email address. Email addresses aren't an inherent property of a person, but if we only need to store a single address per person, it makes sense to put it directly on the person class. So far so good. Next, we could add a method to send emails to that person, and now our cohesion just took a nosedive. The person class now has to know about how to send emails. We could pass in a reference to an email service in order to hide the low-level details, but we've definitely crossed a line here. Just as our front end can ask the user for their full name, maybe some other part of the system could just ask the person for their email address, and then that part of the system could worry about sending the emails. If giving the person class an email address property is okay, then what about physical address and phone number

properties? They're kind of the same, right? But something feels wrong here. More than half the properties on person aren't really properties of a person anymore. The phone number I could let slide for now, but the address fields, that seems a little off. Am I going to give the person class the ability to format the physical address too for when we send out flyers? The person class is losing its sense of cohesion fast. It's taking its first steps down the road to becoming a grab bag of loosely related information. It would probably be better if there were a separate class for address information, so let's divide that off. There, the address class is only concerned with address stuff, and person is only concerned with person stuff. Both are now highly cohesive. A person may have an address, but it's not who they are. Here's another little detail about Mermaid's class diagrams. Just like I added the relationship between person and IValidatableObject, I can add a relationship from person to address, and I can even specify the cardinality of that relationship by labeling the ends of the arrows in quotes. So here, one person can have 0 or 1 address, so they can have an address, but they don't have to. In real life, a person could have multiple addresses, and all we'd need to do is change the notation so that instead of 0 to 1 addresses, a person can have many addresses. We could continue and divide off the phone number and email address properties into their own types as well so that a person could have multiple types of each, but, you get the point. It's better to have multiple smaller, highly-cohesive classes than a few larger classes with low cohesion.

Coupling and Cohesion

Coupling is a measure of how much one class or object is dependent on another. Classes that leverage other classes become logically coupled to them. The more ties you have between them, the stronger that coupling becomes. The problem is that changes to the dependencies can bubble up and require changes in the consumers. The more of these connections a module has to others, the more vulnerable it becomes to these ripple effects. We commonly talk about coupling in terms of it being tight or loose. Tightly coupled code is more brittle, loosely coupled code is more flexible, but what does that mean? Well, let's look at a simple example, logging. Personally, I'm a big fan of Serilog. I like the wide variety of logging destinations it supports, I like its configurability, in general, I just like the way it works, but that doesn't mean I want all of my code to know that it's what I'm using. Most of my code just needs a logger, it doesn't care what logger, and it definitely doesn't want to be responsible for knowing what makes it different from other loggers. The configuration of the different destinations, or

syncs, doesn't matter to most of my code, it just wants to log an occasional progress or error message. We get around this problem by defining a common interface that multiple loggers can all implement, decoupling the requirement from its implementation. This is, of course, vastly simplified from the real-world implementation, but you get the idea, right? When my code needs to log messages, it only needs an instance of something that implements ILogger. It doesn't have to be the one from Serilog, and it doesn't have to be my own custom implementation, anything that satisfies the interface will do. The logger implementation has become interchangeable. I'm only dependent on the interface. I'm now loosely coupled to any particular implementation of ILogger. Returning to our Person example, the more I break up Person into separate classes, like Address, the more relationships Person will have to those other classes. Each class I divide off is one more thing I'm connected to, and that's fine where it makes sense. For instance, Person is now tightly coupled to Address in this example, but that's okay because they're part of the same object model. I don't need multiple implementations of an Address interface, at least not now. It's a judgment call. When you're designing your object model, pay special attention to those types that you suspect might need multiple implementations and consider defining an interface to describe them.

Nouns, Verbs, Adjectives, and Entities

Many object-oriented programming introductions will teach you to think of the components of your system as parts of speech, with your classes being the nouns and your methods being the verbs, which represent the different actions those classes can take. This is a pretty simplistic metaphor, but it's a decent way to start thinking about things when you're designing a system. Start by listening to the client or subject matter expert describe what they need the system to do. Take notes, and jot down the words that you hear most often, like customer, order, shipment, or supplier. They are most likely your business entities, which is just a way of saying that they're the most important objects in your system. Not every class represents an entity. A customer is an entity, but a logger isn't. They're both nouns, but one of them represents a business concept, the other's a service, a tool to help us build the system. It's still significant, but the client you're building a system for probably has no interest in it. Certainly no one's going to argue that string would qualify as an entity. Methods are verbs or verb phrases. They're named for the actions a class can take. Here are a few methods that you might find on an order class. You can ship it, cancel it, complete it. You might also calculate the order's total cost.

Less commonly, you'll hear that interfaces are adjectives, and in a lot of cases that's true. Interfaces describe a combination of properties and methods that combine to describe a capability, such as being serializable, or innumerable, or disposable. For very simple interfaces ending in able, this metaphor holds up pretty well, although it can start to break down for things like `INotifyPropertyChanged`. The parts of speech metaphor is simplistic and kind of naive, but it's still a decent way to categorize your thoughts. If you want to think beyond the simple categories, though, yes, order is a noun and it's also an entity, but it also provides functionality. It's not just data, it's what can be done with that data,

Summary

In this module, we looked at the difference between object-oriented programming and object-oriented design. We identified the four pillars and five SOLID principles. We saw how to use Mermaid to sketch out some basic UML diagrams and then used it to illustrate concepts such as cohesion and coupling. And Finally, we looked at a common approach to recognizing and categorizing system components as parts of speech. In the next module, we'll dive a bit deeper into the four pillars, talk about each one individually, and start seeing how those concepts translate to C# source code.

The Four Pillars

Introduction

Now that we've had a high-level flyover of object-oriented design in general, let's circle back for a closer look at some of the more important concepts. In this module, I'll start building out actual code, but I'd like to point out that it won't all be perfect, not at first. I'm going to build out the classes that make up the demo a piece at a time, refining the design as we move through the upcoming modules. That means that there will be some intentionally simplistic or naive choices made up front, specifically so that they can be replaced later on. The most fundamental foundational principles of object-oriented design are known as the four pillars. These are the language and design features that define object orientation. We'll start with the concept of abstraction.

Abstraction

Abstraction is basically just a model hiding its internal complexity and presenting a simplified interface to its consumers. Remember that module here is just a general term for a chunk of code or a functionality. It applies at multiple levels, but the idea is the same. You don't need to know how a method does its work in order to use it. As long as it does what it says, it's free to implement that behavior however it wants. You don't need to know about the private members of a class, it's none of your business. The reason for wrapping up our code into objects to begin with is so that we can break large problems up into smaller pieces, solve them once, preferably in a well-organized way, and not be bothered with them again. The alternative is just to do everything inline and write piles of unmaintainable spaghetti code. Abstracting away implementation details also gives you the opportunity to make changes to that implementation if a better solution is found in the future, so long as you don't change the module's interface. I've used the word interface several times already in this course, but let's make sure we're clear on two very different uses of this one simple word. There's interface, the C# keyword we're all familiar with, which defines members that a class must implement. It's a way to formally define the general concept of an interface, which is what I'd like to focus on right now. For a method in your code, the interface is just that method's signature. It's the list of parameters, their types, whether or not they're required, and the return type of the method itself. It's the face of the method that the consuming code sees. At a class level, the interface is the collection of exposed fields, properties, constructors, and methods. It's the combined surface area of all of the class's members taken altogether. At the assembly level, it's the combined surface area of all of the classes it contains, an entire library or API. One of the classic examples used to illustrate this concept is a car, something we're all familiar with. All cars present a very standardized user interface. There are variations, of course, for things like right or left-hand drive, manual or automatic transmission, but for the most part if you know how to drive your car, you know how to drive any car. This example is a classic because it's such a relatable, real-world example of something we're all familiar with. Let's start with an absolutely bare minimum implementation of a car. The Car class has properties for its make and model. It also has more transient properties like the position of the steering wheel, throttle, and brake pedals. It exposes methods for starting and stopping the engine. These properties and methods would presumably be hooked up to various appropriate key, wheel, and pedal components that make up the user interface of our car. If you were referring to this diagram in order to learn how to use the Car

class, then the only information you would have available is what you see here, and that's as it should be. The diagram makes no promises about how those members will behave, and that makes it a great illustration of abstraction as a concept. I know what the car does without having to know how it does it. Let's implement this in C#. We have a couple of string properties for the make and model. Make could have been a reference to a separate manufacturer class if we wanted, but let's just concentrate on the Car class for now. Next, we have three floating point properties for steering, throttle, and brake positions. I'm using floats since these are analog controls. These values could represent an angle or a percentage or a position within a range of motion. And Lastly, we have the Start and Stop methods, which return a Boolean indicating the car's status. They would presumably turn the power to the engine on or off, but how that happens is a mystery because it's been abstracted away behind these two methods. While I have a general understanding of how an internal combustion engine works, I certainly couldn't build you one. I don't need to know the inner workings of the engine to drive the car, though. The public-facing interface of the Car class hides, or abstracts, those details away and presents a simplified set of controls for its user, me. The engine-specific controls are reduced down to the start, stop, and throttle position members. We could add on more features to our car's interface for setting the gearshift position, locking and unlocking the doors, reading the fuel level, etc., but this will do for now. Now that our Car class is implemented, let's write some unit tests to exercise it. There's not much active code yet, so all we can really do is test that the Start and Stop methods work. I'll add a unit test project and then a class, called `Given_a_Car`, and use its `SetUp` method to create an instance of the Car class. I'll add another class, inheriting from `Given_a_Car`, called `When_the_Car_is_Started`. And then in its `SetUp` method I will call the `Start` method and remember the return value. And finally, I'll add a test to check that that value was true. I'll then do something similar for when the car is stopped, and there we have some basic tests. Not everybody organizes their tests this way, but I have found that as things scale this works well for me. And If we run the tests, we can see that they all pass. Here's our first object-oriented design tip, make it a routine habit to step back and look at your code as a consumer. What's visible from the outside? Did you simply expose everything as public just because it's easy? It's tempting to fall back on public fields, properties, and methods for expediency, especially when you know that you're the only consumer of a module, but try to resist that urge. Make members private or protected whenever possible. Make them internal if they aren't meant for consumption outside the assembly that they're written in. The less visible your code

is, the more freedom you have to change how it works. Every detail that you expose is a detail that someone out there might decide to leverage, including you, and that makes it one more detail that you can't just change in the future. It becomes a promise, a contract between your code and its consumers.

Encapsulation

Encapsulation, which is sometimes called data hiding, is to information what abstraction is to behavior. A class potentially knows a lot more than it's telling you. Not everything gets exposed to the outside world, and that's part of how it achieves abstraction. Throttle position used to be a very simple concept in the early days of internal combustion engines. You simply open the fuel valve up a little more and the engine would speed up as a result. Things are a lot more sophisticated than that these days, which is how modern cars can achieve their impressive level of fuel efficiency. The timing of the fuel injection and the firing of the spark plugs is all controlled by a computer called the Engine Control Module, or ECM, even though the user is still presented with a simple gas pedal. The ECM monitors and keeps track of all kinds of information from the position of the crankshaft to the engine temperature. It monitors the ambient air temperature and takes that all into account when deciding on the appropriate air/fuel mixture at the time, whether that's warming up on a winter morning or cruising at freeway speeds. None of these measurements appear on the dashboard though. As a car user, you couldn't possibly be in charge of the intricate timing of the events happening under the hood thousands of times a second and still perform your normal duties of actually driving the car. It's a similar concept to abstraction in a way. The ECM's public-facing interface is simplified to expose only the information that's needed upstream, and to keep the rest private, the ECM encapsulates the information it needs to do its job. It's no one else's business. For all intents and purposes, the ECM is a black box. Let's add one to our Car model now. We call this composition, building complex systems out of multiple smaller ones. Each component is still a class in its own right, but together they add up to something more. I've also removed `ThrottlePosition` as a property of the Car class and replaced it with an instance of a `Throttle` class and an instance of a `ThrottlePositionSensor` on the ECM class. The car, as a physical object, still has a throttle of course, but the ECM is responsible for keeping track of it now. We'll do the same for the other ECM values in a minute. First, though, notice that there are some items that are common between the two classes. The `Start` and `Stop` methods exist on both

classes. When you tell the car to start, it'll pass down or delegate that call to the ECM, which does the actual work. The Car class still exposes those members to its caller, that's you, but because it had already abstracted those methods, it's free to change their implementation without affecting its callers. The ECM has also abstracted away its implementation details, as well as encapsulating the information it needs to do its job. Let's see what this would look like in C#. Notice that we didn't pass all of the Car class's responsibilities down to the ECM. In this example, the ECM doesn't care about the brakes or steering wheel position, so the Car class doesn't delegate those functions to the ECM. Moving the throttle responsibilities to the ECM was an important step, though. When we encapsulate data, we want to move it as close to the code that needs it as possible. Our car, as a general concept, doesn't actually care about whether it has a ThrottlePositionSensor or not. It's an item on the bill of materials, sure, but it's only the ECM that actually uses the thing. Here is a finished Car Class Diagram. The car has instances of two new BrakePedal and SteeringWheel classes, which I'm not going to diagram out. The ECM has grown some additional sensors to gather the information it needs in order to do its job, but they're not exposed as part of the Car class's public interface. Like I said at the beginning of this clip, abstraction and encapsulation can seem similar. Abstraction has to do with hiding the complexity of the implementation, whereas encapsulation has to do with the location of the supporting information. In order to write tests that illustrate encapsulation, I'll first need to encapsulate something. That doesn't necessarily mean that the information has to be hidden, but it makes for a good test, so here we go. I've cleaned up and refactored a little, which I won't make you watch, but here's the gist. Instead of the Start and Stop methods directly returning whether the car is running or not, I've created a private status field and a readonly IsRunning property on the ECM. Only the ECM itself can change this private field, but IsRunning makes the effect visible in order for us to test against. I've updated the existing tests to start the car and check its IsRunning property, and similarly for when the car is stopped. Even though the underlying data is completely hidden from the outside, our tests can still verify that calling Start starts the engine like it did before. To sum up, keep the data as close as possible to the code that needs it, and if nothing else in the system needs that data, make it private so it's not even visible to the consuming code. This stops other parts of the system from altering data that's outside their area of responsibility.

Inheritance

If you've done any coding in C#, you already know what inheritance means at the language level, but let's step back and think about it as a general concept. There are features that a car has simply by virtue of being a car. It has properties like a make and model, it has user interface elements like a steering wheel, gas and brake pedals, doors, etc. If we think of the car like code, then somewhere there's a base definition of car that all car models inherit from. Plenty of people own cars, but nobody owns just a car. They own a specific make and model of car. Let's create a specific type of car to illustrate this concept. First, we'll mark the Car class as being abstract. Then we'll add a specific type of car, which I'll call the Globomantics S1 sedan. There aren't a lot of changes to the diagram other than the fact that our S1 sedan inherits from Car, which is now an abstract class, but there's more to see in the C# implementation. Note that this is not an ideal design. We wouldn't want to have to create an entirely new class for each different model of car, but right now we're talking about inheritance, and this illustrates that idea. Our new S1 class is a specific kind or subclass of Car now. As such, it has all the features that you would expect any car to have, like an Engine Control Module. S1 can even override members of the base class in order to make itself more specific as needed. Different subclasses can extend the same base class in different ways to achieve very different kinds of cars. For instance, if our company wanted to be out on the forefront of automotive technology, they might decide to start manufacturing electric coupes in addition to the traditional sedans already in their catalog. For historical purposes, our EngineControlModule class won't be changing names, but an electric car has no engine, so the implementation would be vastly different. We'll make EngineControlModule an abstract class as well, and create two new ECM types, one for internal combustion engines called the IceControlModule and one for electric cars called the ElectricControlModule. Since an electric car has no crankshaft or exhaust system, components for those systems would exist only on the IceControlModule class. The ambient air temperature could still be useful information for an electric car, so we'll leave that on the base ECM class for now. :Let's look at the code. I've rearranged things a little so that each model of car specifies the type of ECM it uses in its constructor. The two ECMs still expose Start and Stop methods, but they work in completely different ways, with the IceControlModule cranking a starter to get the engine going and the ElectricControlModule simply closing some relays. Common elements like AirTemperatureSensor and ThrottlePositionSensor remain on the base class since they're common to both ECM types. I'll have to fix up the tests from earlier. Since Car is now an abstract class, I've changed the existing tests to work

against a specific type of car, the EC1 coupe, and then duplicated them to work against the S1 sedan. The tests can now be made more specific in the future to test things that differ between the models or to verify the things that should remain the same. Inheritance is a major key to code reuse. In this example, you can see that all of the pillars we've covered so far are in play. The two ECM types abstract away their inner workings and encapsulate their state, or the information they require to do their jobs. Together, these two concepts mean that the car's user doesn't need to be aware of any of those details because they simply aren't important to the operation of the vehicle. We could take this concept further by defining a base vehicle class that would define the features common to cars, boats, planes, and motorcycles, but I think you get the idea. Let's sum up. Be on the lookout for commonalities between multiple different classes that are variations on a theme. Extract those commonalities into a base class so that there's just one implementation of the items those classes have in common. When you find members that are specific to only some types, move those members into those types and out of the base class. Avoid the temptation to build base classes that know how to behave as multiple subtypes. A base class should not be aware that its subtypes even exist. And now we have all the pieces we need to move on to the fourth pillar.

Polymorphism

The fourth and final pillar is polymorphism, which means many forms, and it means that subtypes can be used in place of their base types. For instance, a piece of code that was written to accept an instance of the Car class will gladly accept instances of our S1 sedan or EC1 coupe. It will accept multiple different forms or types of Car. As long as it's a car, it's fair game. The classic test for polymorphism is the IS-A test. S1 inherits Car, so that we can say that S1 is a car. Inheritance is one way to achieve polymorphism, but it's not the only way. Your car isn't just a car. It is an entertainment center, it's a cell phone charger, it's a portable shelter. Your car can serve a number of different roles, and the ability to think about it from those multiple perspectives is also polymorphism. For example, if you take your car camping and the sky opens up threatening to wash the tent away, you can always retreat to your car, recline the seats, and wait out the storm there. Just as there are multiple kinds of cars, there are multiple kinds of shelters, so if all you need is a shelter, then a car or a tent will do just fine. C# only lets you inherit from a single parent class, though, but you can implement multiple interfaces. In this case, when I say interface, I mean the C# interface keyword specifically. I'll create a

Shelter interface and implement it on the Car class. We can also implement that interface on a new Tent class. Even though tents and cars are two vastly different kinds of objects, both can be used wherever you need a shelter, so C# interfaces are another way to achieve polymorphism. Our existing tests already demonstrate this. If you noticed, when I split up the existing car tests to create the separate EC1 tests and S1 tests, the system under test property is still declared as a car even though each Test class instantiates a different kind of car. Polymorphism is everywhere, and so taken for granted that you may not even notice that you're doing it. When you're designing a system, take time to divide off the various responsibilities. Don't just create a single class that knows how to do everything. It would be easy enough to create a single Car class and code it to do everything a car would ever need. It would still only ever be a car, though, and it's the only Car class your system has. Splitting out commonalities through inheritance or through the implementation of interfaces means that your code is free to accept multiple types of cars, but also that car can be viewed from multiple different angles in order to satisfy multiple requirements.

Summary

In this module, we looked at the concept of the four pillars, what they mean and how they should affect your design decisions. We looked at abstraction, hiding the implementation details to simplify using a module. My advice was to try to reduce the publicly exposed surface area of a module as much as possible. Encapsulation, hiding or at least moving the data as close as possible to the code that uses it. Again, my advice was to expose as little as possible. Inheritance, sharing common code in a hierarchy of objects by defining classes as specializations of other classes. You should extract your commonalities wherever possible. And polymorphism, the ability to look at multiple objects in terms of their commonalities, or a single object from multiple angles. You should divide your class's responsibilities up so that each class becomes a multifaceted entity that can be viewed from multiple different angles. In the next module, we'll look at the SOLID principles and see how they each relate to one or more of the pillars.

The SOLID Principles

Introduction

The four pillars may define object orientation, but they can be kind of obtuse and they don't really lead your design decisions very well. They tell you what's possible, but not how to get there. What we ended up with at the end of the last module is viable code, but it could be better. There is plenty of advice out there and many object-oriented design principles that offer guidance on how to think about and structure your code. They will, in general, overlap one or more of the pillars. In 2000, Robert Martin, generally known as Uncle Bob, identified what he considered to be the top five most important design principles. After some jockeying for position, they were arranged into the acronym we now call SOLID. There are entire courses on just the SOLID principles, and I won't be covering everything in nearly that kind of depth here. If you want to dive deeper on the SOLID principles, you can check out Steve Smith's Pluralsight course, [SOLID Principles for C# Developers](#). This module will serve as a general introduction to these five principles, though, before you head off to explore more deeply. In this module, we'll work through the SOLID principles and see how they influence the design of our project. Hopefully you'll see that they exert much more influence over design decisions than the pillars do alone. We'll start by going back to a naive implementation of the Car class. It still has all the capabilities from the end of the last module, but represented in a single flattened class. I've also introduced an EngineType property and matching enumeration this time because there have been rumors of a possible hybrid model in the works. I'd like to take this opportunity to point out another mermaid syntax item. If you look at the Start and Stop methods, you'll notice that they don't have colons between the method name and its return type in the source view, but Mermaid will still display colons in the diagram view. Why are methods declared differently than properties? Beats me, but if you actually put a colon there, the return type will disappear from the diagram entirely. Mermaid definitely has some idiosyncrasies that you just have to get used to, and this is one of them. So this is our starting point. We'll spend the rest of this module refactoring this class according to the SOLID principles and see where we end up compared to the previous module.

Single Responsibility Principle

The original definition of the single responsibility principle is that, "A class should have one, and only one, reason to change." Or sometimes it's phrased as, "There should never be more than one reason for a class to change." This has led to a lot of confusion over the years. What exactly is a reason to

change? This principle has been extended and clarified multiple times since it was first coined. But at heart, it's about cohesion. I prefer to define the principle this way, "Every module should have a clearly-defined job." Note that I'm expanding from the very specific word class to the more general term module here because this principle can apply at multiple different levels. The responsibility of each module ranges from highly conceptual at the assembly level to tightly focused at the property or method level. Continuing with the car analogy, the entire demo assembly so far represents the world in which our cars exist. Other vehicle types may exist in this world, as well as dealerships, garages, and owners, but it's the automotive world we're talking about. The feeding schedule of the local zoo animals doesn't fit with this theme, and therefore doesn't belong in this assembly. The car class represents the idea of a car. You could stretch that definition to include trucks and vans, but boats and airplanes would belong in their own classes. At the innermost scope, the car class's properties and methods are each concerned with a single aspect of the car instance that exposes them. Looking at our flattened Car class, though, you can see that it's currently responsible for everything, from simple properties like make and model, to the methods that start and stop the engine. Car has to keep track of absolutely everything. It has lots of jobs, lots of reasons to change. If we still want to handle electric cars, then the Car class will have to implement code to handle both types of engines. The Car class has to know how to do everything all by itself. That's not good. Car doesn't have a single responsibility. It has all the responsibilities. We should do something about that. Just like in the last module, we'll break off some of the Car class's responsibilities to other classes. Let's start by separating out the simple items, like SteeringWheel, BrakePedal, and Throttle classes, as well as the various sensors, although we're not going to diagram them out here. Next, we'll extract an EngineControlModule class once again. And move the air, crank, and exhaust sensor references there. We'll leave the throttle on the Car class since it's part of the user interface, but we'll need to add a ThrottlePositionSensor on the ECM so it can see what's happening. We will still expose the IsRunning property on the class, along with the Start and Stop methods, but the plan is to delegate their responsibilities to the ECM class. The end result should look like this. Let's take a look at how this principle would be implemented in C#. Let's turn these simple float values into instances of sensor classes again. So, that's the AirTemperatureSensor, the CrankPositionSensor, the ExhaustOxygenSensor, and finally, the ThrottlePositionSensor. I'll let the refactoring tools create my new classes to silence these errors. We don't need to fill them in right now; they just need to exist. And then I'll introduce our

EngineControlModule property on the Car class. I'll use the refactoring tools again to create the actual class file, like I did with the sensors, and then I'll move the EngineType and assorted sensor references there. Finally, I'll copy the IsRunning property and the Start and Stop methods to the ECM. When we're all done, we should be at a very similar place to the last module, except that we still only have one Car class. The Car class is not specific to any particular car model yet, but we'll come back to that later. The ECM class, on the other hand, has an issue. The ECM class still has more than one responsibility. It still has to behave differently depending on what kind of car it's installed in. In the case of a gasoline powered car, it cranks the starter to get things going. In an electric car, it closes some relays. I should extract a BaseEngineControl module class like I did in the last module, but I'm going to stop here because I want to tackle this in the next clip instead. This is where I would normally create tests to demonstrate the idea we're discussing, but there's just no logical way to write automated tests for broad concepts. There's no assertion for counting responsibilities. What we can do, though, is divide up our tests so that they match our new class structure. All I've needed to change here is to provide an EngineType value when creating the ECM instance. Other than that, the tests continue to work. The single responsibility principle overlaps with both the abstraction and encapsulation pillars, in that each module or layer represents a single concept at varying levels of detail. The Car class has a single job or responsibility, and that is to represent the idea of a car as a whole. Not every little implementation detail of a car, but the overall collection of other classes that make up that general concept. The more details it takes on, the less focused it becomes on that concept. By dividing off smaller concepts into other classes, like the EngineControlModule, the Car class can remain focused on the big picture, which is its single responsibility. You'll find that the SOLID principles frequently overlap not just the four pillars, but each other. Multiple SOLID principles will often push you in the same design direction, and that's the case here. The single responsibility principle already wants me to split up the EngineControlModule class because it has too many jobs, but the next principle will want me to make a similar change for an entirely different reason, so I'll save that for the next clip. Following the SOLID principles is really easy. Each principle encourages or discourages one particular thing, and the name usually tells you what that is. For the single responsibility principle, this is simple. Examine your classes looking for individual responsibilities or jobs. In the original terms, you'd be looking for reasons to change. Each time you find one of these, it's potentially another class that you could extract. Just like in a real-world car, our EngineControlModule class is just a component. It can

be updated or replaced independently of the rest of the vehicle. Break up large problems into multiple smaller ones, each responsible for a single aspect of the whole system.

Open/Closed Principle

The open/closed principle, as stated by Bertrand Meyer in 1988, says that, "Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification." Uncle Bob rephrased this more succinctly as, "You should be able to extend a class's behavior without modifying it." We shouldn't have to update the Car or EngineControlModule classes every time we introduce a new model of either. They should both be extensible, so we can create new specializations on top of them when needed. Every time you modify a class, there is the potential to introduce a bug, even if your change was itself a bug fix. You've taken a tried and tested class that's currently out in production and you've introduced changes to it. Those changes could have unforeseen consequences. Let's pick up with our single EngineControlModule class from the previous clip. We already know that we should break it up because it currently has more than one job, but the open/closed principle gives us an entirely different reason to do this. If we look at our class diagram again, we have our single EngineControlModule class, which has to fill the needs of both gasoline and electric cars. One day the word comes down from management that the company is getting into the hybrid business as well. That means we're going to need to add a third branch to our switch statement that both closes some relays and cranks the starter. This seems like a simple enough change, but now we have to update any tests that relied on there being only two possible values for EngineType. Don't get me wrong, we still have to write tests for each variation anyway. But because all the behaviors live together, we may have to reevaluate and adjust some of our existing tests. Not only that, but the QA department is now complaining that we've invalidated the manual testing they'd completed just last week. Now they have to start all over again in order to comply with some obscure regulation or another. It would be better if we could simply add a new ECM type without having to touch the existing ones at all. Let's see how to do that now. I'll extract a base class using Visual Studio's refactoring tools and move the common elements to it. So that's the AirTemperatureSensor, the EngineType, and IsRunning properties, the Start and Stop methods, and the ThrottlePositionSensor. Finally, I'll make the new base class abstract because just like the base Car class from the last module, we'll only ever instantiate a subclass of EngineControlModule. All I need to do is mark any of the properties or methods abstract, and the

newly generated glass will be made abstract as well. That leaves the crank and exhaust sensors behind in the original class, so I'll rename this class to be the `IceControlModule` for internal combustion engine. I'll rename this manually rather than using the refactoring tool, so you'll see why in a minute. I'll copy the new `IceControlModule` to create an `ElectricControlModule`. Then I'll override the `Start` and `Stop` methods so that each one performs the appropriate actions for its type. So, the `ElectricControlModule` will close some relays, and the `IceControlModule` will crank the starter, which would get the engine running. To further emphasize this, I'll rename the `backing` field in each class to be more specific. Finally, I'll rename `EngineControlModuleBase` to simply `EngineControlModule` so that the existing code and tests won't know the difference. Now it's time to fix the tests. The `EngineControlModule` is abstract now, so we can't just instantiate an ECM and feed it the `EngineType` anymore. Instead, we'll need to instantiate a specific kind of ECM for each of the two car types. And if I've done this all correctly, the tests should all still pass. And now I can truly show off the open/closed principle in action. We're going to add our new hybrid version of the S1 sedan called, predictably, the `S1H_____`. To prove there's nothing up my sleeve, let's close all the open files. I'll copy and paste the `IceControlModule` class to create a new `HybridControlModule`. I'll expand the `Start` and `Stop` methods to use both the starter and the relays so that the electric motor is ready to help out, and then copy the `relayStatus` field from the `ElectricControlModule`. I'll update the `IsRunning` property to check them both. And finally, I'll create a new set of tests for the hybrid scenario. Of course, real cars are far more complicated than this, but all I need to illustrate here is that the implementation details of these three classes are different, even though their exposed methods and properties mask this from the outside world. We'll give the tests one more run, and there we have it. I implemented an entirely new kind of `EngineControlModule` without having to modify any of the existing classes. I didn't have to make any changes to the base class in order to create an entirely new variant. That's what the open/closed principle is all about. There's one more thing we should probably do while we're in the neighborhood. The base `EngineControlModule` class is abstract, and you're expected to derive other classes from it, but our three ECM subclasses are meant to be the end of the inheritance chain, at least in this demo. I can make this absolutely clear by sealing them. If the company wanted to start making diesel-powered cars as well, then I could make the `IceControlModule` abstract as well and derive separate `GasolineControlModule` and `DieselControlModule` classes from it, but three variations is plenty for now. The open/closed principle maps to the abstraction, encapsulation, inheritance, and polymorphism

pillars. Our base `EngineControlModule` class dictates that its descendants must provide an `IsRunning` property, along with `Start` and `Stop` methods, but it doesn't say how they should work internally. Each ECM class exposes these three members, but they've each abstracted away the implementation details and both encapsulated and abstracted the supporting data. Remember that encapsulation doesn't necessarily mean that the data is hidden, merely that it's located in the class that needs it most. Through these three members, we were able to extend the base class without having to modify it in any way. In this example, we ended up with three classes that all inherit from the common base class. You don't have to leverage inheritance to satisfy the open/closed principle, but it's pretty common, so I'll say it overlaps that pillar. And finally, the fact that the `Car` class only knows about the base `EngineControlModule` class illustrates the polymorphism pillar. `Car` can operate using multiple forms of `EngineControlModule`. The Markdown Editor extension that I've been using to create all of our diagrams is a great example of this principle at work. The extension consists of some code that either inherits a specific base class or implements a specific set of interfaces so that Visual Studio knows how to talk to it. The key thing is that nobody had to alter Visual Studio to make this happen. Visual Studio just had to provide the appropriate extension points to make it and other extensions possible. When you're designing the classes that make up a project, try to sort their members into two categories. There are the bedrock pieces of shared code that belong in your base classes. These are the parts that apply to broad categories and rarely, or never, change. And then there are the parts that apply only to certain cases. These are the specializations built on top of that bedrock, and these specializations are what you provide extension points for. Be on the lookout for code that significantly alters its behavior, such as with that switch statement in the single ECM class from earlier. All code has to make decisions, but try to recognize when a class should be broken up.

Liskov Substitution Principle

The Liskov substitution principle is the only one of the solid principles named for a specific person. Barbara Liskov first defined this principle in purely academic terms as follows. Let $\phi(x)$ be a property provable about objects x of type T . Then $\phi(y)$ should also be true for objects y of type S , where S is a subtype of T . Did you get all that? I didn't at first, either. But all it really means is that derived types must be substitutable for their base types. In other words, if I write a method or a class that uses an instance of `EngineControlModule` it must continue to work when I give it an instance of

any of ECMs subtypes, whether that's the internal combustion, electric, or hybrid version. We don't even have to make any changes to any of the diagrams or even the code to demonstrate this since we already have three subclasses of EngineControlModule handy. The Car class has a property that is declared as the base EngineControlModule class, and we've already populated it with the different types in previous tests. Just to drive the point home, though, I am going to create another Test class, which I'll call EcmTests. In that class, I'll create a helper method that takes in an EngineControlModule, calls the Start method, and then checks the IsRunning property. I'll expand on that by calling the Stop method and checking the IsRunning property again. Finally, I'll create Test methods that leverage the helper method to test the three different ECM types individually. The important thing here is that the TestEcm method is unaware of the differences between those types. And if I run the Test suite (Working), we should have all green lights. Any of the three ECM types can satisfy the requirements of this helper method. In case you're thinking that simply inheriting from the base class is all you'd need though, not so fast. The Liskov substitution principle does not just say that I can pass in descendants in place of their ancestors, it says that descendants have to be substitutable for their ancestors. In other words, it all still has to work. I'll modify the Hybrid type a little bit here to illustrate the point. Suppose that, for some bizarre regulatory reason, hybrid cars required you to manually engage the batteries before you could start the engine. Maybe you have to flip a switch on the dashboard that closes our fictitious relays. I'll expose the RelaysClosed property on the HybridControlModule and then remove the part of the Start method that was setting it before. If I run the tests now, even though I call the Start method, the IsRunning property still remains false because it expects both the relays and engine to be engaged. It's a contrived example of a bad design. HybridControlModule still inherits from EngineControlModule, and I can still pass it in wherever an ECM is required. Structurally, it's still a reasonable substitute for the base class, but it doesn't work. At its core, the Liskov substitution principle is just polymorphism, usually implemented via inheritance, but you can see that it's more than just checking off those boxes. You can't just say I've inherited from a base class, therefore, I've achieved polymorphism, therefore, I've satisfied the Liskov substitution principle. To satisfy the principle, you should be able to use any subclass in place of its superclass, a descendant in place of an ancestor, and expect it to work. It can work in different ways, but it still has to work. This is what makes the solid principal so useful. They push you toward making better design decisions in a more intuitive way than the pillars alone. Don't get me wrong, the changes I've just made go against some of

the pillars as well. The HybridControlModule has broken some of its abstraction and encapsulation. Implementation details are leaking out into the classes consumers. The client or consumer of this class would have to be aware of its differences and treat it differently than the other ECM types, but this was only done to illustrate a specific point. While you should write tests for each individual subtype that exercise their differences, you should also write tests to ensure that each of a type's descendants can perform the job of their common ancestor. In my example, it was making sure that calling Start and Stop on any descendant of EngineControlModule would achieve the same result, even though we know that they're wired differently internally. Subtypes should provide all of their base type's functionality, so consider writing tests to ensure that all the subtypes actually work in place of their base type. One thing to keep an eye out for is if you ever find yourself throwing a NotImplementedException because a type is just like the others, except for that one detail that isn't. The Liskov substitution principle doesn't say it's impossible to do this, It says you shouldn't, so don't.

Interface Segregation Principle

The interface segregation principle states that, "Clients should not be forced to depend upon interfaces that they do not use," which is also sometimes stated as, "Make fine-grained interfaces that are client-specific." Once again, I think the official definition needs at least a little explaining. The word client here means a module's consumer, the other code that uses it. It's the "forced to depend" part that always caused me trouble, so let me try to clear that up. Let's suppose you are working on an aftermarket remote starter system that would let you start any car remotely. If our Car class exposed five different methods, but the starter system only uses the Start and Stop methods, is it really being forced to depend on the other three methods? No, but that's not really what this principle means. The starter isn't being forced to do anything at all with those other three methods. The problem is that it's been forced to depend on a type that has the other three methods in order to get to the two that it wants. It's tightly coupled to the Car class now, so you wouldn't be able to use the same starter system on other types of vehicles. Actually, remote starting a plane is a horrible idea, scratch that. You wouldn't be able to use the same starter on other things with engines, like a generator. That seems like something you'd want to be able to start remotely, right? The starter only needs to be wired up to something with Start and Stop methods, something startable and stoppable. Remember when I talked about the parts of speech metaphor in module two? Well, here we have two adjectives staring right at

us. And adjectives mean interfaces, and that's what this principle is about. I don't want to get too hung up on interface with a little i, the C# keyword, versus Interface with a capital I, the general concept. You could create an object hierarchy to satisfy this principle without using the interface keyword at all. But for practical purposes, we're talking about the keyword. Let's build our remote starter now. The starter is a completely separate class from the car. If done correctly, the car doesn't even need to be aware that the starter exists, and the starter doesn't need to know that it's attached to a car. All they need to agree on is the Start and Stop methods, and that's what we'll create an interface for. I won't be creating separate startable and stoppable interfaces. Being startable implies that you must also be stoppable; otherwise, how could you ever be started again, right? I've seen too many people slap a dozen or more interfaces on a single class describing each and everything it can do separately. Like all principles or patterns, try to have some restraint. Be pragmatic. For example, .NET's ICollection interface has multiple properties and methods on it. They didn't break out a separate IHasEnumerator interface and an IHasCount interface, and so on. Properties and methods that go together can be defined together. The important thing is that they collectively define a single ability, that they are highly cohesive. We just need to implement this interface, this time on the Car class itself. And then we'll implement it in the C# source code. It should come as no surprise that we'll do this by simply implementing the interface on the Car class. Because the Start and Stop methods were already implemented on the Car class, we don't need to do anything else here, so we can build our remote starter class now. It's a simple class with only a reference to some device that implements the IStartable interface, and then the Start and Stop methods, which delegate to that device. Since we need the device in order to be able to do anything with it, I'll just take it as a constructor parameter, similar to how the Car class takes an EngineControlModule reference. Let's update our existing Car class tests to operate against the IStartable interface instead. We'll still need to check the instance of the car to make sure that it's running afterward because IsRunning isn't part of the IStartable interface, but the act of starting and stopping the car is now performed against the interface, not the class. And if I add a Generator class, like we talked about earlier, I could run these same tests against an instance of the Generator class, and they'd never know the difference. And now we can test our new RemoteStarter class, similarly to how we tested the various ECM types earlier. I have two tests, each of which create an instance of something startable, either a car or a generator. The tests then create an instance of a RemoteStarter, passing it the device. Finally, we call the Start and Stop methods on

the starter and make sure that the device has been started or stopped. It won't exhaustively test each different type of car EngineControlModule here, but you can see the benefit of implementing that interface. The RemoteStarter class is totally unaware of what it's attached to. As long as it implements the IStartable interface, it can use it. The interface segregation principle maps most closely to the polymorphism pillar. We established multiple forms of the EngineControlModule class through inheritance. Here we've defined multiple forms of things that are startable through the implementation of interfaces, which looks a lot like inheritance if you squint. When you're designing a system, try to divide up and group its abilities. The properties and methods of the class can be grouped into sets of related functionality represented by interfaces. Keep the members in those interfaces cohesive at a micro level and assemble them into classes that are cohesive at the macro level. Those interfaces let you leverage polymorphism across object types that aren't directly related through inheritance, but are related through their similarities expressed as interfaces.

Dependency Inversion Principle

That brings us to the final SOLID principle, the dependency inversion principle. It, like the other principles, has been described in a number of ways. But the most common definition is that high-level modules should not depend on low-level modules; both should depend on abstractions. Or more succinctly, depend on abstractions, not on concretions. At the code level, what this boils down to is that a class shouldn't depend on other specific classes. A RemoteStarter shouldn't instantiate the car itself. You don't get a free car with every RemoteStarter purchase. Nor should the car instantiate the starter. They should both agree on an interface. And that's all either one of them should be directly aware of. Something else should be responsible for introducing them to each other. In our car example, we have an IStartable interface, which is implemented by the car and consumed by the starter. So it's the starter that should be provided with an instance of something that's startable. There are a couple of ways to accomplish this. The first way is through property injection. Each RemoteStarter instance would need to be provided with an instance of something startable. So some third party would have to construct a car first and then pass it to the starter before the starter could be used. The second and more common method is through constructor injection, which is what this code was doing originally. So I just need to revert these changes. Our RemoteStarter class could require an instance of something startable as a constructor parameter. Now you can't even create an instance of

RemoteStarter without giving it something to start. I've mentioned before that the SOLID principles overlap each other, and this is a prime example. The work done to satisfy the previous principles has already set us up for success with this principle. A lot of developers will be confused by two similar terms. On the one hand, there is the dependency inversion principle. On the other hand, there is a dependency injection and more specifically inversion of control containers. They're not the same thing, but they do tend to travel together. Dependency injection and inversion of control containers are just tooling that facilitates following the dependency inversion principle. They are an implementation detail. I won't be implementing an IoC container here because that's an entire topic unto itself. Henry Been has an entire course on dependency injection in C# here on Pluralsight. I can at least describe their function though. An IoC container gives you a way to declare just once up front how a particular class or interface should be instantiated. In the case of an interface, you declare which class you want to provide when another class requires that interface. In our car example, we could say that whenever some class requires an instance of IStartable, it should be given a car. The IoC will then construct the car along with any components that it requires, subcomponents of those components, etc. And that's basically the job of an IoC. It takes the job of constructing objects out of your way. Back to our example. We won't need to write any new tests here either, but let's examine the previous clip's test through this new lens. In order to test the starter, we needed to give it something to start. So we passed in an instance of Car to the constructor. Then we passed an instance of a Generator and checked that it still worked as designed. Because the dependency was passed in from the outside, the starter isn't dependent on any concrete class to fulfill that dependency. It's only dependent on the abstraction defined by the IStartable interface, so it remains loosely coupled and flexible. The dependency inversion principle mostly overlaps the polymorphism filter with some inheritance thrown in for good measure. A consuming class only needs to declare what it needs, not which specific class should fulfill that need. This is commonly accomplished by asking for an interface on the consuming class's constructor. Try to design your classes in such a way that they ask for what they need rather than building it themselves. Take those requirements as either constructor parameters or as properties to be filled in before they're used. Consider the use of an inversion of control container to help ease the burden of development. This will keep your code loosely coupled and flexible.

Summary

I hope this module has helped you see that the solid principles aren't just a different set of names for the four pillars. They're more than that. The pillars may describe things that your code can do, but the solid principles tell you why that's a good idea. They can each overlap multiple pillars, and each other, to encourage better designs. You may often find yourself being pushed in the same direction by multiple pillars for entirely different reasons. In this module, we looked at the five solid principles. The Single Responsibility Principle says that each module should have a clearly defined job. The Open/Closed Principle says that you should be able to introduce new functionality without having to tear into your existing classes every time. The Liskov Substitution Principle says that subtypes have to be usable in place of their base types. They can't just look like them at a superficial level, they need to work. The Interface Segregation Principle says that a consuming class shouldn't have to know everything about the classes it uses, only the details that it actually needs. And finally, the Dependency Inversion Principle pushes us toward loosely coupled code by letting external forces determine the specific types in play at runtime. In the next module, we'll start looking at some common design patterns and see how they can help you recognize and solve common problems more easily.

Introduction to Design Patterns

Introduction

Now that we've established a baseline for recognizing and evaluating object-oriented designs in general, we can start to build on that foundation. Traditionally, we might walk through some examples of real-world applications like a storefront or a blogging engine. The idea is to build up your experience so that you have the tools handy in the future when similar situations come up in your own projects. That's a useful concept, but you have to recognize experience from your past that relates to a current problem. You have to mentally separate the pieces from the finished system, and that can be difficult. That's where design patterns come in. Design patterns are a similar idea, but broken up into small chunks that describe a well-known solution to a specific problem. They're defined in isolation without context so it's easier to recognize them for the building blocks that they are. They're a shorthand way

of specifying chunks of reusable code so that architects and developers can simply mention them by name instead of describing the concept from scratch each time they occur. Many times when someone mentions design patterns, they're likely referring to a group of patterns that were documented by Eric Gamma, Richard Helm, Ralph Johnson, and John Vlissides, known collectively as the Gang of Four in their book, *Design Patterns: Elements of Reusable Object-Oriented Software*. The book is a collection of 23 common patterns that solve problems you are likely to encounter at some point in your career. Like the SOLID principles, this is not a finite list of all the patterns that will ever exist. Also, like the SOLID Principles module, there are entire courses on just this one topic, so I encourage you to watch them as well. Kevin Dockx has a course on implementing design patterns in C#. In that course, he covers each of the 23 Gang of Four patterns in depth. I won't attempt to repeat all of that here, but we will pick and choose a few to get you started.

Singleton

Let's start with the simplest pattern there is just to establish how we'll talk about the others. If you check out the starting branch for this module, you'll notice that I've organized all the demo files so far under a CarWorld folder and matching namespace to make room for us to talk about design patterns. The singleton pattern is used when you want to ensure that there is one, and only one, instance of a given class shared by the entire application. They act as a kind of global variable and are commonly used to control the creation of shared resources, like logging. It's easy to implement because there's almost nothing to it. It only takes a static property, a backing field, and a private constructor. And here's what it looks like as a diagram. In UML diagrams, the \$ suffix means that a member is static, so it belongs to the class as a whole and not an instance of that class. You can remember this because the dollar sign looks like a modified S for static. Making the constructor private means that you can never create an instance of that class from the outside. The class can make one from the inside though, and that's just what it's going to do. The implementation is just as simple as the concept. I'll create a new class, which I will cleverly call Singleton, then I'll add a private static field to hold the one and only Instance of the single class that we're going to allow. Next, I'll add a public static property to return that Instance, instantiating it the first time it's used, and then returning the same instance forever after. Then I'll add a private constructor, which will block the default constructor so that outsiders can't create an instance of the class. And that's it, mostly. This is a viable Singleton class and will work in simple

situations. But it has a thread safety problem. If two threads call the `_instance` property at the same time, the second thread could call the `if` statement before the first thread has a chance to set the `_instance` property. You could accidentally create two instances, although the second thread would overwrite the value set by the first thread. This can be addressed in a few different ways. We could put a lock around the entire `if` block so that only one thread can even be asking about the `_instance` field at a time. That would have an impact on the performance of every single call to the instance property though. We could add a static constructor and set the backing field from there. Static constructors fire off the first time someone even mentions the class and are guaranteed to be thread safe. At this point, we could get rid of the backing field and just use an auto property to make things even simpler. Since C# 6, we've had the ability to initialize auto properties in line, which is also thread safe, so we could even ditch the static constructor in favor of a property initializer. These last three versions guarantee that the property will be initialized by the time you try to use it, but the runtime could initialize it earlier. Although the Singleton pattern doesn't prohibit you from instantiating the instance before anyone asks for it, it's generally considered bad form. If the singleton is an expensive and rarely used resource, you might want to ensure that it actually waits until it's needed before starting one up. We can address that too. Ever since C# 4, we've had the Lazy type, which takes a function that says how to create the value the first time it's called. And as a bonus, the Lazy class will take care of all those thread safety issues for us. All we need to do is reintroduce a backing field, but make it a Lazy of Singleton, and then initialize it with a lambda that creates the instance. Because the lambda function lives inside the class, calling the private constructor from here is just fine. Now the instance won't be created until someone asks for it at runtime. It's thread safe, and it's still only a few lines of code. So why did I go through all these different iterations for such a simple class? Well, to illustrate that even in such a simple pattern, there's plenty of room for interpretation as long as you stick to the goals and behaviors of the pattern itself. The original 23 Gang of Four patterns were published in 1994, six years before C# was even announced, so it's hardly surprising that a modern C# implementation might differ from the original examples. The important thing is that when you say singleton to another developer, they know the behavior and general shape of what you're talking about. It's technically possible to inherit from a Singleton class, but that can get confusing, and you'd rarely want to do that, so I'm going to seal my Singleton class to prevent that. In order to prove that we get the same instance every time, I'll add a simple integer property to our Singleton class and set it to a random Value when the instance is

created. (Working) And now let's test it. I won't make you watch me type this out, but here's the finished class. It retrieves the value of the Instance property twice and checks that the value property on each copy has the same value. If the test passes, it does a simple `Console.WriteLine` so that we can see what the random value was on that run. And if I run the test, it passes, and we can see that the values were indeed the same random value. If I run it a couple more times, you can see that it's a different value each time. And there we have it, the simplest design pattern there is. When someone says singleton, you know exactly what they mean, even if there are some variations in the implementation. It's a shorthand phrase that's now part of your vocabulary, and that's what design patterns are all about, so now we're ready to move on to larger, more complicated patterns.

Builder

Whenever a class instantiates another class, it becomes tightly bound to that other class. For example, here is a piece of code from the four pillars module where the constructor for the model S1 sedan class builds its own instance of `IceControlModule`. This line is hard-coded to that type, and it's all it will ever create. In order to follow the dependency inversion principle, your code should build as few of its own dependencies as possible. There are several design patterns that are concerned with instantiating objects. They all work in different ways, and which one you choose depends on the situation. These are known as the creational patterns and include singleton, which we just looked at, factories, prototypes, and builders. I'm a big fan of the builder pattern and use it pretty extensively when writing unit tests. You can see my course, *Creating Maintainable Contexts for Automated Testing*, for more information on using builders in this way. But the pattern applies to regular code as well. It's meant to offload the construction of classes, typically business entities, to a separate Builder class so that the consuming class can focus on what it needs rather than on the details of its construction. The basic builder pattern looks like this. The first class we need is the Client, sometimes called the Director. This is the consumer of the builder, the code that needs what the builder will create. Then we have a base Builder class, although the original Gang of Four description makes this an interface. We'll inherit from that to create another class called the `ConcreteBuilder`. Its actual name will change from one implementation to another. Maybe it's an `OrderBuilder`, or a `HouseBuilder` or a `PizzaBuilder`. This is the class that will construct the actual object at runtime. And finally, there's a `Build` or `Construct` method that returns you the actual object you wanted. Instead of instantiating an object

yourself, you are enlisting a specific ConcreteBuilder to do it for you. You're still instantiating the Builder though, so how is this any better? Well, for one thing, you've centralized the knowledge of how to build a particular kind of object in one place, instead of repeating it all over your code base. This diagram is very generic, and like the other patterns, it's only meant to show the bare-bone skeleton of the pattern. There is no one true Builder implementation. You can add features to make your life easier or to take advantage of your chosen language's features. Let's build a real one in C# so you can see what I mean. We'll start with a very plain Product class. It has a single property called Name. This could just as easily have been a House class or a Pizza class, but we'll keep it simple for this example. Next, we'll need a Builder. I'm going to create a base Builder class that takes the place of the builder interface in the original Gang of Four design. I'll make it abstract, generic, and give it a protected auto property to hold the instance being built. Finally, a public Build method will return the finished product to the client code when its ready. Now, I can use this base class to create a ProductBuilder class. That's the ConcreteBuilder from the original diagram. At this point, the class won't do much. If I were to call the Build method, I'd get back a null because we've never set the value property to anything yet, and since I marked it as protected, I can't do it from the outside. Let's add some tests as we go along to keep an eye on the ProductBuilder's progress as we build it out. This test should pass in its current form. A Builder that only returns null isn't very useful, so I'll add a couple of methods to create more useful builders. The first returns a fresh ProductBuilder configured to return a product of type A. The second returns a product of type B. This is a very simple version of another pattern called the factory method pattern, and I like to use it rather than defining entirely separate ConcreteBuilder classes such as Product A Builder and Product B Builder. This is just a personal preference of mine, but it amounts to the same thing. Remember, named patterns are just a starting point for your own code. If I copy my existing test, I should be able to create tests for my two new customized ProductBuilder instances. When I ask for a ProductBuilder for Product A, then that's the kind of product instance it will build, and likewise for Product B. You may be asking yourself how this is any better. I still have to call the correct method on the correct class for the type of product I want, so aren't we still tightly coupled to it? Well, we're slightly less tightly coupled to it at this point, but we can still do better. Instead of two separate methods, let's make one method that takes a parameter and returns a Builder of the appropriate type based on that. I'll create a ForProduct method and then a switch statement based on the value of the productType parameter. If the parameter is A, then we'll

return the value of the existing ForProductA method and then I'll do the same for the value B. If the client passes anything other than A or B, I'll throw an ArgumentException. I could have taken the productType parameter and simply passed it down to the ProductBuilder, but here's where you start to see the value of having Builders in the first place. The ForProductA and ForProductB methods make it obvious to consuming code like tests which product types are valid. This new ForProduct method will also restrict me to building the products that I've defined ahead of time, but it does it based on a string now. Our coupling just got a little bit looser. Let's update the tests. In the interest of time, I'm not going to break this up into one assertion per test. The important thing is that the ProductBuilder can now build product instances based on a string parameter. That parameter could be based on user input or stored in a database. Our code base as a whole still instantiates its own dependencies, but we've centralized that knowledge in one place, the Builder. This example is still pretty simple and flat, though, so it's time to take it to the next level. Let's say our products have two components called LeftComponent and RightComponent. I'll use the refactoring tools here to create the Component class, and then I'll make it abstract so we can make multiple kinds of components. Our ProductBuilder class will need to expand a bit to assign the correct components for each productType. Product A uses Component1 on the left and Component2 on the right. Product B uses Component2 on the left and Component3 on the right. And here's the payoff, none of my existing tests needed to change. They were asking for either a Product A or a Product B, and they're still getting that. They don't need to know about which product types get which component types. The Builder takes care of all of that for them. Let's update the tests again. I'll just add some new assertions to the existing tests to check the types of the Left and RightComponents on the resulting instances, and the tests should still pass. The interesting thing to notice here is that in these last tests I'm now building a multi-layered object hierarchy based solely on a string value. That's pretty cool, but what if we need something slightly different than a standard product? What if we need to customize things? Most Builder implementations will provide you some way to alter the object before calling the Build method, usually by exposing a series of Add or Set methods for each property. Personally, I like to make it possible to chain these methods together in order to customize the object in a kind of fluent API. Let's add some methods to the ProductBuilder class to override the Left and RightComponent values. I'll add two methods called WithLeftComponent and WithRightComponent, and use them to set the appropriate properties on the product we are building. Now, client code such as our tests can start with one of the predefined

product types and then make changes to it by adding calls to the `WithLeftComponent` and `WithRightComponent` methods. The resulting system is very flexible, but it doesn't have to stop there. Maybe components could have their own subcomponents. We could continue to build that knowledge here in `ProductBuilder` or we could divide it off into a new `ComponentBuilder` class to encapsulate that knowledge and add customization methods to those builders. The same goes for subcomponents of those components, and so on, building out an extensible and flexible system of Builders for each layer of our object hierarchy. At this point, we could even expand the input string to act as a kind of bill of materials, with each layer reading its parameters from the string before passing it down to the next builder in the chain. I could come up with a dozen variations on this pattern that solve the same puzzle in a dozen different ways, but I think we should stop here. The builder pattern is a highly flexible way to build objects without having to hard-code that knowledge into every single place an instance of those objects are needed. It centralizes the knowledge of how to build the objects into a single place in your code, and that's what object orientation is really about, defining tools for the rest of your code to leverage. There are many variations on the pattern and they may not match the original Gang of Four diagram exactly. There is no mention of `With` methods on this diagram, so am I doing something wrong? No, patterns are not set in stone. They're starting points for your own code, and you can expand on them to fit your particular needs. Your builders may look different than someone else's, but they're still recognizable as builders. Anyone familiar with the pattern will have a pretty good idea of how they fit into your overall system, even if the details change from project to project.

Strategy

Before we wrap up, let's look at one more common pattern. You may find yourself needing to tackle a similar problem in multiple ways. The classic example of this is a discount calculator. Maybe a customer has some coupons, as well as a loyalty discount and there also happens to be a sale going on. How can you decide on the proper discount to apply to the customer's order? That's where the Strategy pattern comes in. This pattern lets you define multiple implementations of a piece of logic and then decide which one of them to use at runtime. We start with a simple strategy interface. In the classic example, this interface is simply called `Strategy`, and it defines a single operation which we'll call `Algorithm`. You would change these names to something more descriptive and specific to your situation. Next, we'll need something that uses the strategy. This is usually called the `Context` class,

but you can think of it as the client or the consumer of the strategy. And it has some method on it that does some kind of work that requires the strategy. And lastly, we'll need one or more concrete implementations of the strategy. Whichever of these two classes is assigned to the Strategy property on the Context class, will determine the strategy that gets used at runtime. Because they both implement the same interface, they're interchangeable, and the Context class doesn't need to know or care which one it has been given. A common extension to the original Gang of Four pattern is to give the consuming class a collection of concrete strategies instead of a single instance. The context could then run the various strategies in order of precedence and stop at the first one that returns a result. Or it could run all the strategies and pick the one with the best result. Either approach could be more appropriate in different situations. Let's see how we could implement this in C#. If you load up the starting branch for this clip, you'll see that I've prepared a very basic Order, OrderItem structure so that we have something to apply a discount to. We'll start to build out our Strategy pattern with an interface, which I'll call IDiscountStrategy. We'll add a method to be the algorithm, which I'll call GetDiscount. It takes an order and returns the discount as a decimal amount. Now let's create a couple different concrete implementations of DiscountStrategy. One, is a coupon that takes a percentage off your highest priced item. That should be easy enough. We'll just implement the interface. Then, in the GetDiscount method, we'll use the Max link operator to get the price of the most expensive item. We don't care about the quantity because this discount is only good for a single item. We'll multiply that by the discount percentage, and that's our result. Of course, in a real-world scenario, the percentage would probably be loaded from a database, but that's outside the scope of this demo. Let's build a second discount strategy that takes a percentage off of your entire order. Once again, it's a simple link expression to figure out the discount. Only this time, we'll sum up the Quantity times the Price of all of the items and then multiply that by the Percentage. We'll unit test these two implementations in isolation before moving on. First, we'll create a sample order. We'll give it two of the most expensive items just to check that it only gives us one discount. Then we'll put a few lower-priced items in the same order and pass the entire order to an instance of the HighestPricedItemDiscount class with a discount of 50% and check that the answer comes back correctly. So that answer should be half of the highest price, which is 10, so our answer should be 5. Let's run this and check the result. It passes, so we're ready to move on. For the WholeOrderDiscount tests, we'll do the same thing. I'll just copy the existing test, change the class name slightly, and then

the file name, and then the method name. I'll change the type of the strategy we're testing here, and make the discount 15%. The result should be 15% of the order total, which is 32 and a half dollars or pounds or whatever currency you want it to be. So that amount should be 4.875. I'm not going to worry about rounding that off here. That's the point-of-sale system's job. I'll run this new test, and we should be all green. Great! Our discount strategies work individually. Now, let's test them together. Suppose we have a point-of-sale system, and it knows what all the current sales are. I won't build an entire system here. Instead, I'll just build what we need to demonstrate the discount strategies at work. So we'll need a list of the active discounts, which can just be a simple list of `IDiscountStrategy` instances. And then I'll need a method to pick the best one for any given order, which, again, can be represented as a simple link expression. I'll add a new test here for the point-of-sale system. In a real system, we would presumably retrieve the active promotions from a database, possibly through the use of a builder pattern. But for this test, we'll just load them up directly. I'll add the same two discounts from earlier, so that's a `HighestPriceltemDiscount` of 50% and a `WholeOrderDiscount` of 15%. And we should still expect that discount to be \$5. And If we run the tests, that's exactly what we see. The `HighestPriceltemDiscount` is the best choice for this order. But what if the customer bought three of the first item in the order. Now the `WholeOrderDiscount` strategy wins with a discount of 6.375. I could write a second test for this, but I think we've seen what we need here. The point-of-sale system is running multiple different strategies for each order and choosing the best one. You could continue adding more and more strategies. Maybe the `HighestPriceltemDiscount` excludes sale items, while the `WholeOrderDiscount` includes them. Maybe some discounts only apply to orders that include a specific discount code. The point-of-sale system itself wouldn't need to know any of these details though. As long as they implement the `IDiscountStrategy` interface, it can use as many or as few concrete implementations as you give it. In a production application, you could use a builder pattern to construct a list of discounts based on data stored in the database and inject them into the `Discounts` collection. The database would store the active discounts, what strategy they use, the date ranges when they begin and end, their percentages, and maybe even whether they can be combined with other discounts in the same order. The Strategy pattern, once again, is just a starting point. What you build on top of it, that's up to you. Just like the other patterns, there are multiple different ways you could implement the Strategy pattern. Certainly, the original Gang of Four authors could not have predicted that I'd be using link expressions to implement the individual strategies or to pick a winner

from multiple choices. But other experienced developers would recognize what's happening here as being an implementation of the Strategy pattern.

Summary

Design patterns establish a shared vocabulary used by developers to communicate known solutions to common problems. I didn't make a big deal out of it at the time, but I'd like you to take a moment and consider how each of the three patterns we covered relate to the pillars and principles from earlier modules. Singleton is pretty simple. Unless you get into inheriting based singleton classes to make others, it really only intersects the abstraction pillar, since it's hiding the mechanism that hands you back the same instance every time. Since it doesn't make any other promises about what the class does, it doesn't inherently intersect any of the SOLID principles, either. What you do with a singleton class will determine that. The builder pattern encapsulates data and abstracts the mechanism for creating the final product. Each Builder class should be focused on building a single kind of object, and delegate the construction of supporting objects to their own builders. So, there's the single responsibility principle at play. The strategy pattern intersects the most pillars and principles out of the three patterns we covered. Logic is abstracted away into individual strategy implementations, each of which should adhere to the single responsibility principal. These implementations hold or encapsulate the data needed to do their jobs, such as the discount percentage in our demo. Each strategy either inherits from a common base class or implements a common interface, which is tightly focused on a single well-defined task, satisfying the interface segregation principle. These strategies can be freely substituted for one another, satisfying the polymorphism pillar, and the Liskov substitution principle. It's even a kind of inside out version of the open/closed principle, since the client code doesn't have to change in order to use new strategies. A lot of thought went into the choice and design of the original 23 Gang of Four patterns. They form a catalog that developers should become familiar with in order to recognize these solved problems when they come up in your own projects. Once you've learned these patterns, you'll start to recognize the problems they solve, and you you save yourself a lot of time by not reinventing the wheel. Be careful that you don't rely on patterns too heavily, though. Don't fall into the trap of thinking that all programming issues can or must be solved using some combination of the 23 Gang of Four patterns. This is sometimes called pattern fever, or patternitis, and it can stifle creativity, leading to what I call pattern paralysis. When you find a problem that a pattern is meant to

solve, then by all means use it. If you find yourself looking for excuses to use a pattern, regardless of how much you personally like it, you should take a step back and try to resist that urge. If your code only has a fixed set of paths it can ever possibly take, then a simple if or switch statement may be sufficient, and a strategy pattern would just be overkill. Patterns are not a collectible card game. You don't get extra points for using more patterns than your co-workers. In this module, we defined what patterns are, both in general and as a shared catalog. We diagrammed and built out examples of the singleton, builder, and strategy patterns in isolation. We considered how they map to the pillars and principles from earlier modules. And we wrapped up with a warning that while recognizing the need for patterns in your own code is important, you need to avoid an obsession with patterns, commonly known as pattern fever, or patternitis. Patterns are tools, and they are each appropriate in the very specific circumstances they are meant to address. In the next module, we'll take a brief look at a relative newcomer in the .NET world, the concept of explicit nullability.

Nullability

Introduction

In this module, we're going to talk about nullability in .NET and how it affects the design of your classes. We'll start with a brief recap of how we got to where we are now in the .NET framework in the C# language. Then, we'll look at how the compiler and Visual Studio can help you write safer code and avoid bugs related to null reference exceptions. We'll examine our code's responsibilities when it comes to dealing with null values, and finally, we'll look at how you can opt out of null checking and why you might occasionally need to do that.

Nullability In .NET

Let's take a quick recap of nullable behavior in C# over the years. We'll use this simple Person class to illustrate the different behaviors. This nullable directive up top is just forcing this class to behave as it would have prior to C# version 8. We'll come back to that in a little while, but for now you can just ignore that. Since the very first version of the .NET framework, we've always had the concept that an

object reference, like `CreditCard`, might be null. `String` is a reference type, so I could also assign null values to properties like `MiddleName`. I've used this in past projects to represent missing information, things that we simply do not know. As a quick example, let's suppose that I'm working on a project that allows users to self register. I don't want to make users fill out everything before they can start using the system. Their email address, first and last name are all we require for a user to get started. We'll bug them to fill out their profile later on. Until then, we won't have certain information, like middle name, so a null value is appropriate. If that user later filled out their profile and left the `MiddleName` field blank, then I'd record an empty string because that's the information they provided. Obviously, this would happen at runtime, not directly in the class, but right now, I'm more interested in the value than how it got there. Maybe this empty string means they don't have a middle name, or maybe it means they didn't feel like sharing it. Either way, we asked them for this value and they left it blank, so we still know that this is the answer they gave. We didn't have a similar concept for scalar values, like height, though. If I try to assign a null height, I'll get an error. The best we could do to represent a missing scalar value is to store some special value, like 0 or -1. Fortunately, C# version 2 introduced nullable value types, which we represent with the question mark you're all familiar with. So we could finally store null value types, which we could use to represent missing information. This is just a syntactic shorthand, and the type is really an instance of the generic nullable type, which is wrapping your original type. We'll stick with the shorthand syntax, though. And that's how things stayed for the next 14 years. With the release of C# 8 in 2019, we got nullable reference types. But wait, I said that we've always had the ability for reference types to be null. So why are we suddenly saying things can be null that could already be null in the last six major versions of the language? I'll change this directive at the top of the class to enable null reference checking, and we can look at the difference. Now Visual Studio has given me some suggestions for any reference properties that are set to null. It's also adding warnings to the error list. This one says I haven't set the non-nullable email address value before leaving the constructor and that I should consider declaring it as nullable. The code still compiles just fine, though. That's because these are only warnings. Enabling null checking only asks the compiler to tell you if it sees anyplace where you might be incorrectly assigning null to a reference property that wasn't explicitly marked as nullable. Warnings are easily ignored, though, so let's tell the compiler that we want to take these findings more seriously. I highly recommend treating warnings as errors in all of my projects. It forces me to deal with these issues right away so that they don't get left

behind and forgotten. Just right-click on a project and open its properties. Here under the Build group is a section called Errors and warnings. I'll check off Treat warnings as errors, and then close the tab. If I return to the Person class now, you'll see that the underlines have turned red, and I can't build the project anymore. Now, I have to fix these errors before I can move on. I'll take Visual Studio suggestions and add a constructor to set the required fields. I'll mark the CreditCard property as nullable since we don't want to require that upfront. Maybe if a user signs up for some kind of premium service plan, we'll ask for it then. This is done using the same question mark syntax as the shorthand for nullable value types, so it's easy to remember. I'll mark the MiddleName property nullable as well, and our class should look like this. Everything's happy, there are no errors, and I can build the project again. But we're not done here. All we've done so far is to ask the compiler to proofread our code. We're not safe yet, as you'll see in the next clip.

Guarding Against Null Values

Now that our code can specify whether properties and parameters are nullable, let's see what this looks like to other code that uses ours. What happens when we try to pass a null into a non-nullable property? We'll start with a simple test that creates a new instance of the Person class, passing an null for the emailAddress. You'll see the that the null value gets underlined and a warning appears in the Error List. Treating warnings as errors is a per-project setting, and I never changed the test project's behavior, so we can still compile despite this warning. And this test even passes. This is an important point about declaring nullability. Marking a property non-nullable doesn't guarantee safety. You don't know whether external code will honor your wishes, so it's best to assume that it won't and guard against null values just like you did before C# 8. The quickest way to do this is using throw expressions, introduced in C# 7. First, I'll convert the EmailAddress property to a full property with a backing field, then I'll add a null coalescing operator and throw an ArgumentNullException if value is null. If your property setter needs to do some extra work before setting the backing field, such as raising events, then you'll need to be a little more verbose about it. Here I'm using the new ThrowIfNull method on the ArgumentNullException class. No more messing with nameof or discard variables. It's one neat concise statement now. However you do it, just remember that it's still your code's responsibility to guard against nulls from the outside, and that also means that its your code's responsibility to document that behavior. To be polite, you should always add XML documentation to

point out any exceptions that your code might throw and why so that consumers of your code are aware of the consequences. At this point, we've documented the fact that we won't accept nulls for the emailAddress property and warned other developers what will happen if they try. Through the new nullable directives and code decorations we've even enlisted the help of the compiler to give hints when consuming code even looks like it might be trying to pass a null value. We still can't actually stop other developers from passing a null, but we've done just about everything we can to warn them not to. By now, you've certainly noticed that the Person constructor has an error. It's complaining that the non-nullable emailAddress backing field hasn't been set yet. We are not allowed to leave the constructor without having set this non-nullable field to some value, but we're setting the property, which sets the backing field, so what's the big deal? Well, the static code analysis that drives these nullable field warnings is smart, but it's not that smart. You and I both know that by the time we leave the constructor either the backing field will have been set or an exception will have been thrown, but the compiler just doesn't see that. In future versions the C# compiler may get better at figuring this out on its own, but for now we'll need to set this field to something. We could explicitly initialize the field to something like an empty string. We know that the constructor is going to override this value anyway, so what's the harm? Apart from a negligible performance hit from allocating a string here, not much, but there's another way. We could explicitly set the backing field to null, but now we have an error again because we said that the backing field isn't allowed to be null. And here's where we're going to do something a little shady, we can use the exclamation point, or null-forgiving operator, to tell the compiler, trust me, I know what I'm doing, and silence this one error because we know for a fact that this field will be set by the time we leave the constructor, if we leave the constructor.

Project Settings and Nullability Contexts

Nullable reference types have the distinction of being one of the few new language features that are turned off by default. If you update an older project to C# 10 or higher, you won't see any of these warnings. You have to opt in to this new behavior. You could add these nullable directives at the top of every class to enable null checking, but that's easily forgotten and would get tedious after a while. So let's look at a better way. If we open the demo project's csproj file, you'll see this nullable element near the top. Setting this affects all classes in the entire project, although it can still be overridden using the compiler directives if needed. I said this feature is off by default and it is. But the templates that Visual

Studio uses to create new projects have it enabled. So any new project will opt in from the beginning. Since null checking is on for this project, I can remove this directive from the Person class file and nothing should change. If I wanted a particular class to disable null checking, I could still override the project setting here just like I did in the first demo of this module. I can even disable null checking for just part of a class. Suppose for a moment that we didn't have the null forgiving operator. I could surround just this one line with a pair of compiler directives to disable and then reenabling null checking. This has essentially the same effect, telling the compiler that I don't want it to worry about this field being null. If I changed the project-level setting to disable null checking though, then this second line would override that decision, and this class would be checking for null from this point on. There's a third option though. By using the restore keyword, I'm telling the compiler to go back to whatever setting the project says to use. Using the restore keyword means that I'm only overriding the behavior of the lines between these two directives. Now is a good time to talk about nullability contexts. As the compiler is evaluating each line, it's in one of several modes or contexts, which are controlled by the project settings and compiler directives. We've seen the enabled and disabled directives already, and you can think of them as turning null checking completely on and completely off. In reality, there are two separate contexts that can be controlled individually, the annotation context and the warning context, and they do different things. You'll probably never use them on a new project, but they can be useful while transitioning older projects. To see how they work, let's make the person's CreditCard property non nullable again and initialize it with a null constructor parameter. This gives us a warning. And since I have warnings as errors turned on, that warning has been promoted to an error. If I disable the nullability context around the CreditCard class's constructor, that error goes away. You might not have been expecting that. We didn't disable null checking here in the Person class, so shouldn't it still be complaining? That's where the two separate underlying contexts come into play. The annotation context says whether to provide nullability information to your code's consumers, in this case the Person class. CreditCard is no longer exposing any details of its nullability support to the Person class. So, Person is completely unaware that nulls aren't allowed. With nullability checking completely disabled, I could even set the credit card property to null inside the credit card constructor itself and it won't complain. The constructor doesn't care about its own violations, and it's not communicating its requirements to consumers either. If I disable just the annotation context, something interesting happens. Disabling just the annotations means that I still want to be told about my own violations, but I

don't want to bother my code's consumers just yet. You could use this while transitioning a library from previous language versions. Once you've addressed the findings within your own code, then you could fully enable null checking and expose that information to your code's consumers. You can also do the opposite and disable only the warnings. Now my own code is free to break the rules, but I want the compiler to warn its consumers about any potential violations. In this mode, I'm asking my consumers to get their act together immediately, but I don't want any warnings internally. I can only think of one situation offhand where this would make sense, and that's a simple property bag class where I know that all the properties will be set by the constructor. I don't want outside code to set those properties to null, but I also don't want to suppress warnings about each and every backing field individually. Completely enabling and disabling nullability at the project level with the occasional override is all most of us will ever need. But fine-grained control is available if you want it.

Summary

For a module about literally nothing, there is certainly a lot to learn. As a consumer, nullable reference types are all about getting the compiler to help warn you when you might be about to pass a null where it isn't supported. As a library author, it's about warning your own clients when they're about to do the same. Either way, it's about communicating the intent of your code's design to its consumers. Here are the important takeaways. You should enable null checking in new projects so that the compiler can help keep you safe. This is the default for new projects anyway, so that's easy. You should also go back and enable null checking in your existing projects since it can help you find potential null reference problems that you may have missed. Override null checking with compiler directives as needed during the transition period. Don't be fooled into thinking that turning on null checking automatically guards against someone passing you an null though. You are still responsible for checking values at runtime, just like you were before. Check your parameters, throw argument null exceptions, and document them. Warn your code's clients about what might happen if they do pass a null where you've said not to. In this module, we took a brief look at the history of nullable values in C#. We started with nullable value types introduced in C# 2. We continued with nullable reference types, which came along in C# 8. Then we looked at how nullable reference types affect your code as a consumer, how to give the compiler hints when you need to suppress individual warnings, how to enable and disable null checking for individual classes, selected parts of those classes, or even entire

projects. And finally, we saw how to control the two separate null checking contexts. In the next module, we'll look at the fundamentals around type equality, immutability, and how those concepts come together in the concept of record types.

Equality, Immutability, and Record Types

Introduction

As soon as you start passing complex objects around a code base as parameters, you start running into potential issues with side effects. Changes made by one module can affect other modules in unexpected ways, introducing hard-to-find bugs. Functional programming mitigates this problem through the use of immutable objects, but you don't have to change languages to do this. In this module, we'll start with an overview of immutability support in C#. Then we'll take a look at how we define equality for different kinds of objects. Finally, we'll see how those two ideas come together in the form of records introduced in C# 9 and record structs new to C# 10.

Immutability

Let's start by talking about immutability, the idea that an object cannot be modified after creation. We create a lot of classes. We use them to represent our business entities, and we use them as containers for utility methods. Classes are our go-to solution, and we use them to attack just about every problem presented to us. We create a class, throw on some properties, and use it to shuttle data around the system. When we pass complex objects like this around from method to method, we leave our code open to potential side effects. At any point along the chain, a method could make a change to the object without any regard for the object's other consumers. For example, here's a set of classes involved in printing envelopes. We start with a Person record, which contains an Address property. I have a simple test that passes a person to the FormatAddress method on the AddressHelper class. That method returns the person's address, upper-cased, formatted, and ready to print on the

envelope. And we can see that the test passes. But if I put a breakpoint at the end and debug, the problems become apparent. If we examine the contents of the Person object after printing the envelope, you can see that the casing of several Person and Address fields have been changed along the way. As a consumer of the class, we certainly weren't expecting that printing an envelope would make permanent changes to the person's data. But if we look at the FormatAddress method, we can see why. The FormatAddress method altered the properties of the Person and Address instances in place. Of course, the simplest answer to this problem is don't do that. I'm not saying this is well-written code, I'm saying it illustrates the problem. You should only change objects when that's your actual job. It's easy to slip up, though, and accidentally make a change without noticing. Instead of passing the entire Person entity to the FormatAddress method, you could pass just the necessary properties. Without direct access to the Person, FormatAddress can't change its values. Of course, if any of these values were themselves complex objects, then the problem repeats itself and you'd need to unwrap those parameters as well, like we're doing with the Address properties. This works, but the method now has a lot of parameters, and that can get messy and confusing after a while. You could wrap these parameters up in an entirely new object, a pattern called a Parameter object. I'll create a MailingInfo class, including only the properties that AddressHelper is going to need. The FormatAddress method doesn't actually care about the first and last names, so MailingInfo will take the FullName as a single string. The other properties can be copied from the existing Address class. Then I'll update AddressHelper to use the properties of this new object instead of the properties from the real business entities. The AddressHelper method is still trying to update the properties in place, that's still a bad idea, but since we're not passing the original person or address instances, at least they're safe now. MailingInfo is a throwaway object, so we don't really care that AddressHelper changes it, but AddressHelper and anything else downstream don't know that their changes are being thrown away. At the very least, we're not effectively communicating what can and can't be done with this new class. In the last module, we saw how explicitly declaring nullability helps to communicate your code's requirements and expectations to its consumers. A big part of good design is communication, so let's make it perfectly clear that updating the fields of the Parameter object is not allowed. Let's make it immutable. You actually deal with immutable objects all the time and don't even think about it. Strings and DateTimes are both immutable, which you can demonstrate pretty easily. These two variables start off as references to the same instance of string, as verified by the AreSame

assertion. When you update a string, you're really creating an entirely new string and a new place in-memory and throwing out the original. So after the change, they're not pointing at the same string anymore, as verified by the second `AreSame` assertion. Running this test confirms what we already knew, strings are immutable. Immutability is a pretty simple concept, but it has some powerful benefits. Because they cannot be updated after creation, immutable objects are inherently thread safe. After all, if your code can't update an object, then it never has to worry about code on some other thread updating it at the same time. Being immutable also means that you only have to worry about validation at the time an instance is created. Have you ever tried to create an invalid date? Let's create a `DateTime` instance to represent February 29th, 2000, which was a leap year. The code compiles and runs just fine, but if I try to create a similar date in 2001, which wasn't a leap year, it will throw an `ArgumentOutOfRangeException` at runtime. I'll fix up this test so that we're actually expecting the exception to be thrown. You simply cannot create an invalid `DateTime` to begin with, and since you can't modify it later on, you can safely assume that any `DateTime` instance always represents a valid date. Let's fix up our `MailingInfo` to be immutable, too. There are a couple of different ways to accomplish this. I could remove the property setters, since all the values are being set by the constructor anyway. And now if I look at the `AddressHelper` class, you'll see that it has errors wherever it was trying to update the properties of the `MailingInfo` class. The compiler won't let this code even try to update the properties of this immutable class now. In the previous module, I talked about how simply declaring a value as a non-nullable type doesn't guarantee someone won't pass you a null, so let's address that. Like the `DateTime` class, we can validate the parameters passed to the constructor. And now if I try to set any of the properties to null, I'll get the expected `ArgumentNullException` at runtime. Don't forget to document your exceptions in the XML comment header. We've already covered that, though, so I'm not going to do that here. The intent of our design has been clearly communicated to any consumers. There's no mistaking what they can and can't do now. `AddressHelper` has no choice now but to make its changes in local variables. Arguably, this is what the code should have been doing in the first place, but now we have a way to enforce that. Before C# 9, this was pretty much your only choice for how to implement an immutable class. Make the properties read-only and take the values in the constructor. C# 9 brought us a new keyword, though, `init`, which says that a property can only be set by the class's constructor or through object initializers. I can change my properties to say `init` where `set` would normally go, and now I could add

other additional properties without having to add them all to the constructor. As an example since Address2 is optional, I'll remove that from the constructor. Consuming code like our test can still set it, though, through object initializer syntax. For a very simple class where all the parameters are optional, you could even get rid of the parameterized constructor altogether. Use immutable parameter objects to clearly communicate your expectations to other modules. This includes these specific fields required by that module. Rather than just passing an entire Person object out of convenience, our use of the MailingInfo class made it clear which properties would actually be used by the AddressHelper class. The immutability of the properties also makes it obvious what the module can and can't affect. They give you an opportunity to validate the input upfront before it ever makes it to the consuming code. And don't forget to repeat this at each layer of a hierarchy. Don't assume that an entire object hierarchy is immutable just because the root object is. If MailingInfo had any properties that were themselves complex types, you'd need to repeat this process at each layer.

Equality

The most fundamental question we can answer about any type is whether two instances represent the same thing. Knowing whether two variables are equivalent can be very important to business logic. And what you get for free isn't always what you want. Let's compare the two major types of equality and then see how to decide the type used by your own classes. Value equality is easy to understand. Are the two objects equivalent? For simple scalar values like an integer, it's as simple as comparing the two values together directly. What could be simpler? A and B are equal, even though they are clearly not the same variable and we didn't derive the answer in the same way. But what about objects with multiple properties on them like a point? Same result. They're considered equal even though they are not the same variable. This is what we expect from symbol value types, so it's not surprising at all. Reference type of equality is actually even simpler. Two reference type variables are equal if the two references point to the exact same instance of the class in memory. Even though these two instances of MailingInfo are identical in every way, they're still not the same. Classes exhibit reference equality by default. What if we want these two instances to be considered equal though? We'll need to make some changes. In order for our class to support value equality like integer and point do, we need to implement the IEquatable interface. This isn't something most of us do every day, so I'll walk through it. Implementing the interface requires us to implement the Equals method. We know that this instance

isn't null or we wouldn't be here in the first place. So if the other instance is null, then we can immediately return false. Next, we'll explicitly check for reference equality since an instance should always be considered equal to itself. Then we should check that the types are, in fact, the same, in case someone tries to pass in a subtype of MailingInfo. You can also simply seal the types so that we don't have to worry about any subtypes existing in the first place. You can support equality with subtypes if you want, but that's outside the scope of this demo. If we've made it this far, we just compare all the properties together to decide whether our two instances should be considered equal. One benefit of implementing equality yourself is that you get to set the rules. Maybe we don't want to compare all the properties together because some are considered unimportant. We know that Address2 is frequently not provided, so we just won't include that in our equality formula. Unless someone else has the same name, but lives in a different apartment of the same building, it should be fine. I'll update the test to expect the two instances to be equal, even with different Address 2 values. This is the bare minimum you have to do to implement your own equality, but there are several other steps you should take to avoid confusion. What if I had phrased my test differently? What if my test use the == operator? The test fails now because the == operator doesn't come free either. We have to explicitly implement it. Unlike the Equals method, we do need to check directly against null here. When the left and right operands are both null, we'll return true; otherwise, we'll just return the value of the Equals method. Overloading == means you have to overload != as well. We'll just negate the result of the == operator, and we should be all set. If I try to build now, I'll see errors saying that we have to override ObjectEquals and GetHashCode as well. I'll use autocomplete to sketch out the object Equal signature. I'll rename the obj parameter to other because I'm a sucker for symmetry. Then I'll call the regular Equals method passing (other as MailingInfo). Now we should be able to compare our class against other objects that aren't known to be of the same type. I'll use autocomplete again to fill out the GetHashCode signature. GetHashCode should return an integer that represents a hash of the object's values. There's no one right way to do this, and the hash implementation is completely up to you. But the important thing is that for any given set of values on an instance of this class, it must return the same value every time. The values that determine the hash code should be the same values that determine equality since it's commonly used to quickly find items in collections like a HashSet or a dictionary. If you have eight or fewer properties, you can simply use the hash code class's Combine method like this. If your object is more complex, you can add an arbitrary number of values by using

the Add method. I'll add one more test to explicitly check the two instances hash codes against each other. And that should do it. My tests should pass now. That's a lot of work just to check whether two instances of a class are equal. It's important to have this background though so you can understand what's still happening under the covers, even when we start getting some of this functionality for free in the next clip. When you want one of your types to exhibit value equality, make sure you don't do it halfway. If you implement IEquatable, do it all. Implement Equals, and that means both methods, the type specific one and also the one that operates on untyped objects. Implement the == and != operators. And override GetHashCode so that it returns the same value every time, at least for the fields that define value equality for that type.

Records

C# 9 introduced the concept of record types. They're a lot like classes but with some different rules. For one thing, they implement value equality and immutability by default. So all of that messing around with overriding methods and operators isn't necessary anymore, at least not for simple cases. Records have an extremely simple format if you don't need anything fancy. Let's see what the MailingInfo class looks like as a record. That's it. That's the whole declaration of the type. This declaration syntax is just a shorthand known as positional parameters. I can use it the same way as the hand-created type from earlier, although the constructor will require all the properties on it including Address2. If I try to change any of the properties, I'll get an error because records are immutable by default. You can see here that it has been declared with an init only setter. So in just this one line, we get almost all the functionality of our handwritten class from the last clip, but it's a very default implementation. For one thing, I'm now forced to pass Address2 in since it's part of the constructor. The actual issue with positional syntax is that there's no null checking on the constructor parameters, although C# 11 will introduce the required keyword to deal with this. For now, the output doesn't match our expectations, but there was no exception thrown. You don't have to use the positional parameter syntax though. What the compiler really generates behind the scenes looks more like this. We get a bunch of init-only properties and a constructor that takes every property, although the casing may not be what you're used to. If I define the record longhand like this using a more traditional property syntax, I can take back some of the control that we had in the hand-implemented class. For instance, I could remove Address2 from the constructor again. And I can add back in the null checking for the required parameters. If I run the

tests now, I'll see an `ArgumentNullException` for the `FullName` property in our test. I'll put this `FullName` property back how we found it and run the tests again. There's one failure, and it's due to the fact that the free `IEquatable` implementation takes all the properties into account, including `Address2`. You would address this the same way we did with the handwritten class by implementing the `equals` method yourself and excluding `Address2` from the formula. And then you'd implement `GetHashCode` to do the same thing. If I run all the tests now, they should all pass again. Interestingly, I didn't have to implement object equals or redefine the operators, so using a record has still saved us some time. Notice that we're not inheriting or overriding anything here. There is no base record class that this one inherited from in order to make it a record. In fact, records can't inherit from classes other than `object`, and other classes can't inherit from records at all. Records can inherit other records though, which is why I needed to make the `equals` method virtual this time, or I could just seal the type. There's a lot of room for customization with records if you're willing to put in the work. For most cases, the built-in positional property syntax gets you what you need, a lightweight immutable comparable type. Records have some other abilities that we won't go into here. You get an overridden `ToString` method that prints out all of a class's members in a format that looks a little bit like JSON, and you get a `deconstruct` method that's like a constructor in reverse, breaking an object into individual out parameters. These topics don't really fit here, but Roland Guijt has an entire course on C# records right here on Pluralsight. Please check out that course for a deeper dive into all of the abilities of record classes. This is the first time that I've called them that, and it's to distinguish them from record structs which are new to C# 10.

Record Structs

I've been referring to the C# 9 records as simply records, but starting with C# 10, they're more specifically known as record classes to differentiate them from the newer record structs. While record classes implement value equality, record structs actually are value types, just like regular C# structs. You'll often hear that value types go on the stack and reference types go on the heap. That's not entirely accurate, but it's a reasonable enough mental model that we'll keep it for now. In general, structs should be lighter weight than classes. Let's convert our `MailingInfo` class to a record struct to see the differences. We'll start by simply adding the word `struct` after `record`. Right off the bat, we have a problem. Structs can't be inherited, so we'll need to get rid of this `sealed` keyword. The constructor is

also mad that we never set Address2 to anything, but I can do that in line with the property itself. Next, we'll need to change the signature for the equals method slightly. We just changed this record from being a class, which is a reference type, to being a struct, which is a value type. That means that this question mark means something completely different now. In the last module, we talked about how for value types, this is just shorthand for an nullable of MailingInfo. I don't want to compare against a nullable of MailingInfo. I want to compare against MailingInfo. So I'll get rid of this question mark. This null check and ReferenceEquals check no longer makes sense for a value type, and since you can't inherit from a struct, there's no need for this explicit type check, so it can go, too. And now all of the tests should pass once again. So that's it. We turned our original MailingInfo class into a record class and now into a record struct. That wasn't so bad, but we also started from a customized version of the code. What would it look like if we had gone with the positional parameter syntax? Syntactically, it looks the same as the record class. I have to provide Address2 again, of course, and now it's making my two instances not equal in the test because those values are different. There's one important thing to notice, though. The properties aren't immutable. If I hover over the FullName property, you can see that it has a getter and a setter, unless, that is, I add the readonly keyword. Now I'll get an error again for trying to set the FullName property, and you can see that the setter has changed to init. Record structs get one more new advantage over record classes. Record structs get a parameter-less constructor as well, which means I can use initializer syntax for the whole thing and skip filling in the optional fields again. Any properties you don't set will get the default value for that type, which means that reference properties you've declared as non-nullable will, in fact, be null, and the tests will pass with this new edition, verifying that Address1 is, in fact, null. Remember, nothing is validating the properties we said weren't nullable aren't being set to null anyway. An answer for that is coming in C# 11. But for now, if you're just looking for a lightweight PropertyBag to pass between methods, then a record struct defined using positional parameters is perfectly fine. If you want anything more out of it, you may need to take back control by overriding the built-in functionality. There's one more topic to cover before we wrap up here. If record types are, or at least can be, immutable, then how can a method pass on a modified version to some other piece of code? Currently, the AddressHelper class looks like this. We've defined local variables to hold the modified address information. Since these are just local variables, we can do whatever we want with them. This method isn't following the advice from earlier modules, though, because it has two distinct jobs going on. Step one normalizes the

individual properties to be uppercased because that's what the post office prefers for mass mailings. The second half actually formats out the address for printing, and, yes, I know I could have made the two upper calls in-line while appending lines to the `StringBuilder`. But this capitalization here is just a stand-in for more complicated real-world processes that standardize abbreviations and validate postal codes, etc. Let's extract this top section into its own method called `NormalizeAddress`. Using Visual Studio's refactoring tools to extract the method results in this hideous collection of out parameters. I think it would be much nicer if `NormalizeAddress` simply handed us back an updated `MailingInfo` rather than a bunch of individual out parameters. Let's change the return type to `MailingInfo` and get rid of those out parameters. Now, the method can simply create a new instance of `MailingInfo` and populate it with the values we would have returned in the out parameters. Then I'll just fix up the `FormatAddress` method to pull values from the new instance of `MailingInfo`, and this is a classical functional programming style right here. Updates are made to an entirely new copy of the object, leaving the original untouched. And this would be fine, except that I have to pass each and every property to a new copy every time I need to change anything. If this object gets passed through multiple methods that each affect a different subset of the properties, then they'd each have to copy all of them to a new instance. What if I add a new property to `MailingInfo`? I'd have to track down each and every usage and update them all, and that's tedious and error-prone. Let's suppose that our `NormalizeAddress` method only knows how to handle the actual address fields and not the name on the top line. It still has to copy all the fields, so as not to throw out any information, of course. Back here in the `FormatAddress` method, we still need to capitalize the `FullName` property before building the return value because `NormalizeAddress` didn't do it for us. Does that mean we'll have to create another copy of the object by hand? Fortunately, no. Records have us covered. We can make a new copy of a record and only mention the differences from the original using the new keyword, `with`. Here I'm taking the new instance returned from `NormalizeAddress` and then changing only the `FullName` field on that. This means I only have to mention the properties I actually care about. If more properties are added in the future, they'll be copied over to the new instance automatically. I'll update `NormalizeAddress` to use the `with` keyword, too, shielding it from any new properties as well. And if I haven't mistyped anything, then all of the tests should still pass, unaware that anything has even changed internally. You should probably start looking for places in your own code where a record type is more appropriate than a class. They can be used as lightweight substitutes for classes when your needs are simple.

They can make writing code much easier and more convenient. And the fact that you're using them at all informs your code's consumers about your intent. It says that this object is probably immutable, so even though you can override the normal record behavior to create mutable records, you should avoid doing that. It's not what other developers are expecting and it will just confuse people. Use readonly record structs when all you need is a simple PropertyBag, and especially for parameter objects, since being passed on the stack is generally faster than passing a reference to an object on the heap. Again, the stack versus heap mechanic is a little more nuanced than simply saying all structs go on the stack and all classes go on the heap. But if what you have is just a PropertyBag with a small number of simple properties, it's still a reasonable way to think about things.

Summary

In this module, we looked at immutability support in C#, which can allow you to follow a more functional programming model. Then, we took a brief look at reference versus value equality. We saw how value equality works under the hood by writing our own implementation of the IEquatable interface. We implemented all the additional baggage that comes along with that interface such as the == and != operators, the GetHashCode method, etc. Then, we introduced the new record types from C# 9 and saw how they can take over a lot of the burden of creating simple immutable types. Finally, we looked at the new record struct types from C# 10 and how they differ from the older record classes. In the next module, we'll look at some common programming tasks and how they can be solved from an object-oriented angle.

Object-oriented Examples

Introduction

Every project is different. Every situation is unique. No one can give you all the answers to every challenge you'll encounter as a developer. But you can adjust your approach to solving these problems to a more object-oriented mindset. That means building smart objects that encapsulate the data needed to support the logic that they abstract away from the outside world. In this module, we'll

look at this from two different angles. First, by implementing a new type from scratch, an enhanced `DateTime` type that understands offsets to account for time zone differences. Then, we'll look at one of my favorite .NET gems that often gets overlooked, strongly typed configuration, because I think it makes a great case study in object-oriented thinking..

Demo: OffsetDateTime

For a warm-up, let's build a minimal offset-aware `DateTime` type that understands how to compare itself with other instances, even when they're from different time zones or offsets. Imagine you and a friend are performing some work at the same time, but on opposite sides of the world. To you, the actions happen at noon, but to your friend, they happen at midnight. Of course, .NET already provides `DateTimeOffset`, which has far more functionality than what we're going to build here, and the Noda Time library knows more about time zones and their history than you'd ever want to tackle on your own. This is just a warm-up, though, and building it will illustrate the core idea of encapsulating not just data, but the behavior around that data. Our needs are very simple for this one. We'll start with a new struct called `OffsetDateTime` to differentiate it from the built-in `System.DateTimeOffset`. I'd like to make it a record, but that won't work with the interfaces we'll need to implement in just a little while. This new type will contain two pieces of information, a time span representing the offset ahead or behind, let's say, Greenwich Mean Time, and a `DateTime` to hold the actual value. Offset could have been an integer representing hours, but there are some time zones out there that are offset by increments of 15 minutes. It also could have been an integer representing minutes, but a time span will make the date math much simpler later on. I'll add a constructor here that takes the normal `DateTime` components, so that's the year, the month, the day, the hour, minute, and second, and then I'll add one more parameter to take the offset as a time span. If this were a real production-ready class, I would create a series of overloaded constructors here, just like the regular `DateTime` class has, but this is fine for this demo. This seems simple enough, but what can we do with our new type? Well, not much yet. What I'd like to be able to do is compare two instances of my type against each other to see if they represent the same moment in time, even if they're from different time zones. Let's build this one in a test-driven manner. First of all, an instance of our type should be considered equal to an identical instance with all the same values. And if we run the test, it passes. So far. so good. Two instances with different values should not be equal, so let's test that next. And if we run the test, we should still pass.

But what if the second value was for an hour earlier, but one time zone later? For instance, midnight in New York is the same as 11:00 P.M. in Chicago, let's simulate that. Again, I'm not going to bother with the definitions of time zones themselves. There are libraries out there for that. I just want to check that these two values would be considered equal. So I'll make the time an hour earlier, but the offset an hour later, and then compare the values like before. And, of course, that new test will fail because we haven't implemented that functionality yet. Just like in the last module, I'll need to implement the `IEquatable` interface in order to define what equality means for this type. I won't make you sit through all of that over again, so here's the result. As you can see, I've had to follow all the same steps as before. I implemented `IEquatable`, overrode `object.Equals` and `object.GetHashCode`, and then overloaded the double equals and not equals operators. I've also updated the tests to include assertions with the objects cast to `object`. Running the tests succeeds, and we're ready to move on. Testing for equality is great, but the thing about dates and times is that they're relative. We should be able to test whether one instance of our `OffsetDateTime` comes before or after another instance, too. Let's write some tests for that, and then we'll make it work. First, let's make sure that a time in a later offset is considered less than the same time in an earlier offset. And, of course, I'll fill in the greater than version of the same thing. In order for this to work, the classes need to implement the `Comparable` interface, or, more specifically, the `Comparable` and `Comparable of OffsetDateTime` interfaces. If you've never had to implement these before, it's a lot like implementing `IEquatable`, but instead of returning a `Boolean`, we return an integer. If the current instance is considered less than the instance it's being compared to, we return -1. If the current instance is greater than the other instance, we return 1, and if the 2 instances are equal, we return 0. If we want to use our greater than and less than operators, we'll have to overload them, just like we did with the equals operators for testing equality. The end result will look like this. In each case, we're simply applying the offsets to the values and then using the existing `CompareTo` methods from `DateTime` to do the actual work. Nothing to it. I've filled in some additional tests as well. Now let's step back and take a look at what we have. We've defined a new class that encapsulates the information needed to represent a time with an arbitrary offset, like you would get when dealing with time zones. It abstracts away all of the logic for determining equality or comparing values in relation to each other so that the consuming code, like our tests, doesn't have to know how any of that works. In this example, our tests just know that they can compare two instances of our type together the way they would an integer or a regular `DateTime`

value, and that's what this course is all about. We've created smart data, objects that know how to do things. We've created a tool that the rest of our system can use instead of dealing with any of these details directly. Of course, I don't want you to actually use this class because these are already solved problems. .NET already defines a `DateTimeOffset` class that does all of this and more. And if you need actual time zone information, you should look at the Noda Time library. This was just a simple example, a warm-up. Now let's look at a more practical example of software tool building.

Options Pattern

I want to approach the next demo from a different angle. Instead of implementing something from scratch, let's build on top of an existing piece of .NET called the options pattern. Configuration in .NET usually looks something like this. You start with some configuration values in an `appsettings` file. Here I have four values of varying types. You just load your configuration sources into a configuration object, like this. You usually see this in ASP.NET projects, but I'm using a console application here for simplicity, and it works just fine here as well. Once you have them loaded up, you just read them out wherever you need them. Here I'm writing them out to the console window so that we can see that it's working. There are some problems with this approach, though. For one, it won't complain if you give it a bad setting name. Instead, I just get the default value of the underlying type. I will get errors if I try to cast a value to the wrong type, though, so that's at least something. The second major issue I see a lot is that developers end up passing around the entire configuration object so that other modules can pull the values they need from it as well. In some ways, you can kind of consider this object-oriented. After all, there's a configuration object, and it exposes methods to get values from the settings file. It's not very well-encapsulated, though. Every part of the system that needs configuration values would require knowledge of the `appsettings` file structure, what the setting is called, what section it's in, and they're all floating around as magic strings. The next logical step for many developers is to try to cache settings on startup, storing them in global variables or perhaps as properties on the `Program` class itself, like this. You could add some validation code here as well and complain about any missing or invalid settings to ensure that the program is properly configured before allowing it to run. .NET already has a mechanism for doing all of this, though. It's called the options pattern, and it's surprising how many developers have no idea that it's even there. Let's implement that now. I'll start by creating a class to hold the values from the `appsettings` file. I'll add a property for each setting I defined in the

file, so that's StringSetting, IntSetting, BoolSetting, and DateSetting. Notice that they are each defined as the appropriate type for that setting. To silence this complaint here, I'll explicitly set StringSetting to null. We'll load it at startup, so we should be fine. I'll also add a constant to hold the name of the section where these values can be found. In my case, it matches the name of the class, but I found that that's not always the case in real projects. Back in Program, I'll use the configuration object we've already loaded, get a handle to the appropriate section of the settings file, and then load it into our type using the generic Get method. Then I'll change the WriteLine calls to use this new object rather than getting each property inline. And if I run the program again, we should get the same results as before. At this point, we have a strongly typed object with our configuration values loaded into it. This class has a single responsibility, and that's to encapsulate the Settings values. Remember, encapsulation doesn't necessarily mean hiding. Now, our configuration isn't just some shared PropertyBag that everything pulls values from. It's an object, and we can give it behavior. I could add default values for when the appsettings file doesn't contain a particular value. We'd still have to cache an instance of this Settings class somewhere in order for other code to get to it, though. Or, we could take these settings as a dependency wherever they're needed by requiring an instance in the constructor. I could go full dependency inversion here by hiding the settings behind some interface, but that would be overkill for a simple PropertyBag, like this. I'll extract, these Console.WriteLine calls into another class, which I'll call SettingsLogger. It takes an instance of sampleSettings as a constructor parameter and exposes one method, LogSettings, where we'll write out the settings like we did in main. I'll even give it an interface for a good measure, although this is not strictly necessary. And there we have it. The logging takes place in a completely separate class using settings that are passed in as a constructor parameter. The SettingsLogger class has no idea how the settings were loaded in. It doesn't need to know. It doesn't care. It also doesn't care how these four properties are implemented. It just needs an instance of the sampleSettings class, so let's give it one. Back in Program, we could create an instance of SettingsLogger, passing in the sampleSettings instance we have on hand. And it works as expected. But where's the fun in that? Let's create a service provider instance to act as an inversion of control container. This is yet another thing that we take for granted in ASP.NET applications, but not everyone knows that you're free to use it anywhere you want. I create a new instance of ServiceCollection and add my dependencies to it. So, that's the settings themselves and the SettingsLogger class that I want to use them. At this point, I'll need to add a reference to

Microsoft.Extensions.DependencyInjection, but after that, it's just business as usual. All that's left is to pull an instance of `ISettingsLogger` from the service provider and call the method. And if we run the application, we get exactly what we expected. This is pretty clean, but we can still do better. With the addition of another package, we can make our configuration even smarter. I'll add the `Microsoft.Extensions.Options.ConfigurationExtensions` package, and then I'll change the way we register the options. Instead of just registering a singleton instance of our concrete options type, let's use the `AddOptions` method instead. And if I run it now, it all breaks down. We no longer have an instance of our class registered. Instead, what this code registered was something that implements the generic `IOptions` interface. And here we find ourselves on the other side of several object-oriented concepts like encapsulation and abstraction. We don't really know what class is behind this interface or how it works because we don't need to know. We need to take in that abstraction here instead, and that's the dependency inversion principle at work. I'll change my dependency to be `IOptions` of `SampleSettings`, and then I'll set the property to `samplesSettings.Value`, and now everything will work again. So what did this gain us apart from some slightly wordier syntax? It's all about taking the settings that you used to pull out of one big configuration object and breaking them up into multiple smaller smarter objects. It's about injecting those objects as dependencies to other objects. From a more practical viewpoint, though, it got us validation. I can now chain another call on this `AddOptions` call in `Program`. I'll call `ValidateDataAnnotations`. I'll add the `Microsoft.Extensions.Options.DataAnnotations` package, and now I can use the data annotations we're all used to to enforce value ranges, or use regular expressions, or to make certain settings required. You can even write your own completely custom validation code if you need to, although that's well outside the scope of this demo. Let's see what happens when the Settings file has errors. I'll remove the `DateSetting` value, and I'll get rid of this default value in `SampleSettings`. Because `DateTime` is a struct, it can't be null, so I'll have to make it nullable as well. I should get an exception at runtime, and I do, but the error didn't happen where you might have expected. The error didn't happen until I tried to resolve the dependency. We have validation, but not upfront. Our application could be running for quite a while before a dependency like this is resolved. What if this setting was for a function that only runs at midnight? In .NET 6, another extension came along, `ValidateOnStart`. It's in the package `Microsoft.Extensions.Hosting`. Unfortunately, this still isn't the same as saying validate immediately. This extension was created with hosted applications, like an ASP.NET project in mind,

and it won't help us in a console application like this. There are two main approaches to solving this problem. The first is to simply force the evaluation of all the option types at startup. If you're after something quick and dirty, that will do the trick, and I've certainly done that before. A more sophisticated approach is to turn this console application into a hosted service. It's not that hard to do, and it will allow the validation mechanism to work as designed. This isn't a deep dive on hosted services, either, so I won't make you watch me type it all in, but here are the highlights. Main now returns an async task. It calls `Host.CreateDefaultBuilder`, passing in the command-line arguments. It loads up the configuration sources, and configures the `serviceCollection`, which should look familiar to any ASP.NET developers. The only really new part is here at the bottom where it calls `AddHostedService` for a new type, which I've called `ProgramService`, though the name is completely arbitrary. Finally, main calls `RunConsoleAsync` on the host instance, which will start up any hosted services that we've registered. Now let's look at that `ProgramService` class. I've defined it in the same file, but it could live anywhere. The important thing is that it implements the `IHostedService` interface, which defines two methods, `StartAsync` and `StopAsync`. So this is a good example of the interface segregation principle. This interface defines just these two operations in any class, and any project is welcome to implement it and can be plugged into this runtime system. We get full dependency injection support here. So we take in an instance of something implementing the `IHostApplicationLifetime` interface, which we will use to wait until the application is ready before proceeding. We also take an instance of `SettingsLogger`, which we'll use to log the settings to the console, like before. I've moved the call to `LogSettings` inside `StartAsync`, fixed the bad settings values, and that should be everything that we need. And upfront validation should work now. To prove that the validation is actually happening at startup, I'll copy and paste the `SampleSettings` class to make a new class called `OtherSettings`. Since I haven't added any matching settings in the `appsettings` file, I should get an exception at runtime, even though no one has asked for an instance of other settings yet. And there we have it, upfront validation of strongly typed configuration values. If this was all too heavy, remember, that this was only necessary because `OptionsValidation` wasn't written with console applications in mind. If you can live without the upfront validation, then you could skip all this hosted service business, but I had to at least show it to you. Let's look at the `SettingsLogger` class once again. This constructor parameter seems pretty ugly. Why can't we just inject an instance of the `Settings` class directly? Well, the reason we're taking an interface here is that there are actually

multiple choices of interface, and which one you choose depends on how dynamic you need your settings to be. There's `IOptions`, which we've just seen. It will always return the settings as they were at the time the program was started. You can think of it as a singleton instance of our settings. `IOptionsSnapshot` reloads the settings once per request in an ASP.NET application. So you can think of it as a request scoped instance of your settings. That wouldn't mean anything to a console application like this, though. And then there's `IOptionsMonitor`, which is a singleton, like `IOptions`, but it raises events when the settings file changes, allowing you to react to those changes. Before I wrap up here, I'll just say that If you don't need any kind of dynamic reloading, then you actually can register the `Settings` object directly. In addition to the `AddOptions` call, you could register the settings type with the `serviceCollection` as well using this overload that lets you use the service provider to register another service. When someone asks for a `sampleSettings`, build one by calling back in to the service provider, pulling out the `IOptions` instance and returning the `Value` property. I've specified the type here for clarity, but that type can be implied from the return value, so I can remove that. Because I registered this as a singleton, that `sampleSettings` instance will then be cached and returned for every use in the future. You could register the settings as scoped, and it would act similarly to the `IOptions` snapshot. It wouldn't really work for `IOptionsMonitor`, though, since `sampleSettings` doesn't raise events to react to. You may have noticed that there are a lot of log messages at the end of the run now. That's because the bulk of our console app is running as a hosted service now. You could configure a logging service to send these messages somewhere else, or you could just add a call to `ConfigureLogging` and clear the providers collection. And now you'd never even know that it was a hosted service at all. But this was not meant to be a deep dive on every single feature of the options pattern. I just think it's a good example of an object-oriented method of attacking a very common problem that many developers simply don't know about. I encourage you to read Microsoft's documentation on the pattern for more information because there's a lot more that you can do with the options pattern. Hopefully this module has helped to illustrate how even the simplest concepts are opportunities to build smart objects. From comparing dates and times to loading configuration values, what seems like a simple concept is often more sophisticated than it appears on the surface. Wrapping simple data in smart classes can have many benefits. For example, taking settings as constructor parameters makes it easy for unit tests to dictate the configuration values in order to manipulate the system's behavior.

Summary

In this module, we looked at two common problems from different perspectives. First, we built a custom type to add some more intelligence around the comparison of `DateTime` values. Then we built on top of classes from the `Microsoft.Extensions` namespace to make configuration easier to consume by defining options types. First, we bound their values directly in the code. Then we made them available via the service provider. We let the configuration system take over the binding to the `IOptions` interface. And finally, we converted a simple command-line application to a hosted service in order to take advantage of upfront validation of those settings. In the next module, we'll wrap up by putting everything we've learned so far together.

Putting It All Together

Introduction

It's time to take the concepts we've covered in the this course and put them together in an example project or at least as many concepts as we can in the remaining time. We'll start with the four pillars. Where do we want our data to live and what intelligence do we want the resulting system to have? Are there properties that we can extract into base classes so that we can think of related objects in terms of their commonalities? As we build out the system, how will the SOLID principles affect the design? And what existing patterns can we leverage to make that design possible? I'll be taking a very traditional object-oriented approach to the design here, focusing directly on the entities themselves, the concepts that make up the problem domain. This is certainly not the only way to design software systems. Other approaches exist that think about design from a different angle. Julie Lerman and Steve Smith have a Pluralsight course, *Domain-Driven Design Fundamentals*, that may very well change the way you approach designing software. For this module, we'll be building a system to provide quotes for vehicle repairs. We'll focus on the business entities and logic rather than on specific technologies, although I will be using Entity Framework Core in the this example. I won't be doing anything fancy with it though, so you should be able to follow along just fine, even if you don't use it yourself.

Designing the Solution

A local auto repair shop has asked us to build a service-quote system for them. Let's start with a single, high-level description of the system expressed as a use-case statement. As a repair shop owner, I would like an automated system to generate quotes for vehicle repair orders so that I can provide accurate cost estimates for my customers. From just this single statement, we can get a good idea of what we need to build. It summarizes the language and ideas that describe what's most important to our client. I see the verb generate in relation to the quotes, that's the function of the system. It's what we need to accomplish. I also see the key business entities that will make up our object model. I see vehicles, quotes, repair orders, costs, estimates, and customers. For our purposes, we'll treat estimate as a synonym for quote. Let's start with a class diagram to define our business entities. During this stage, it's all about breaking down the information into classes and deciding what information belongs where. If the problem domain is understood well enough, the classes will seem natural and obvious. The shop has customers with some basic contact information so that the shop can call them when the repair is complete. I'm not going to specify property types or visibility on this diagram since we're just sketching out the general shape at this point, and we've already covered that in earlier modules. If I were preparing design documents to hand off to other developers, then I'd need to be more specific. Anyway, these customers have vehicles in need of repair, so we'll want to know some basic information about them as well. This time we won't be modeling the inner workings of the vehicle like we did in earlier modules. Instead, we're modeling only what's relevant to the repair shop, so we'll need basics like year, make, and model so that the technician can find the car and the current odometer reading. When a vehicle is brought into the shop, a mechanic will examine it and create a repair order through some other system. The repair order contains a list of individual repairs to be completed, so we'll want to add those to the diagram as well. Don't worry about the brackets here. Repairs won't necessarily be an array, I just want to capture the fact that there are multiple repairs in an order. Repairs have a Category, a Name, and an associated Price. The shop has created a shorthand code for the repairs they perform, which they currently maintain in a spreadsheet somewhere. Our client would like to continue using these codes, so we'll add a Code property to our Repair class as well. After discussing things further, we determine that the cost of a repair isn't a fixed number, it's the sum of part and labor costs. Each repair has an associated list of parts and a labor estimate expressed in hours. The cost of those parts and the labor rate may

change over time, but the parts themselves and the labor hours don't. We'll need to define the Part type as well. Each Part has a Name, a StockNumber, and a Price. The resulting repair order will be fed into our system in order to generate a quote. A quote is a customer-facing version of the repair order that summarizes the cost of the repair. It holds the part and labor costs and has a reference back to the original repair order for more details if you need them. Quotes are good for 30 days, even if parts and labor costs change, so we'll need a property to hold the expiration date of the quote. The quote will be responsible for the formatting of a report to be signed off by the customer before work begins, so we'll plan to override its ToString method. For this demo, we'll consider a number of items out of scope. We won't be handling parts inventory or ordering from suppliers. We also won't be making the parts vehicle specific such as tracking which kind of brake pads go on which vehicle. We'll only implement a single labor cost rather than charging different amounts for different departments or labor categories. And finally, we won't be taking a snapshot of part and labor costs as part of the quote, the shop will simply honor the original totals until the quote expires, and we'll let the accountants worry about the numbers. And that's our initial set of entities or at least a rough sketch of what we know so far. Entities don't exist in isolation though, so now let's think about the relationships between them. A customer owns one or more vehicles, so that's a one-to-many relationship, which we denote by marking each end of the relationship with the cardinality in quotes. Technically, a customer could have no vehicles, although we wouldn't be doing any repairs in that case, so let's assume they always have at least one vehicle. Each repair order is created for a specific vehicle, which could be serviced multiple times over the years, so that's a one-to-many relationship from vehicle to repair order. A repair order contains a list of individual repairs, so that's another one-to-many relationship. Each repair has a list of parts, but some common parts could be included in multiple repairs, so that's a many-to-many relationship. And finally, since quotes expire, a single repair order could have multiple quotes associated with it over time, so that's a one-to-many relationship from repair order to quote. This phase of the design process is all about talking things out between the client and the developers, as well as between one developer and another. Try explaining the design to each other and look for any spots where the explanation bogs down or gets difficult. Those areas may need more attention and lead to design changes. Take the time to iterate the design until you're happy with it and can speak fluently about it. We still need to decide where to put the logic for generating quotes for repair orders. On the one hand, we could put it in the RepairOrder class and consider a quote to be the

output of a repair order. On the other hand, you could consider a repair order to be the input to a quote, so we could put the logic in the Quote class. In this case, GenerateQuote would be an example of a factory method pattern. These are both valid choices with many of the same advantages and drawbacks, so it's really a matter of choice. I'm going to do something different and define a QuoteService class, though. The client has informed us that many repairs are covered under manufacturer warranties. Warranty coverage will be handled by a separate warranty service, and I don't want my business entities to be in charge of making those kinds of calls. I'll add in a QuotesService class where our quote-generation logic will live. It has a single method, GenerateQuote, which accepts a RepairOrder as a parameter and returns a Quote. I have no reason to suspect there will ever be more than one implementation of QuoteService, but as a matter of convention, I'll define an interface for it. QuoteService will require the use of a warranty service, so I'll add that to the diagram as a placeholder for now, and we'll come back to this later in the module.

Implementing the Domain

Ordinarily, I'd start by creating a brand new solution, but I want to keep all the code for this course together, so I'll just add a new project to our existing demo solution. I'll start with a project where the entities and logic will live. I'll call the project RepairShop.Domain. Let's start by creating a folder to contain the business entities that make up our domain. I'll call that folder Model. We need the Customer class, which will be very simple for this demo. I'll add an integer Id property, but otherwise it's just the three properties from the class diagram. I'll use data annotations to further describe our properties such as setting the maximum lengths of the string properties. We'll talk about Entity Framework Core in a little while, but it's worth noting that it can infer a lot of information without any additional help from us. For instance, this property called Id will become the primary key for the Customer table by convention. And since nullable reference types are enabled for this project, Entity Framework will make use of that to determine that the database columns for these string properties should be non-nullable. At this point, you can expect a warning about those same string properties because they haven't been initialized to anything. As we discussed in the module on nullability in C#, I can initialize these explicitly to null, using the null-forgiving operator. The Customer class will need a property to hold the list of vehicles that they own. Entity Framework Core can use anything that implements ICollection, but I'll initialize this one as a HashSet since no vehicle should appear in the

same collection twice. This could just as easily have been a list. Again, I won't make you watch me type them all out, but if you pull the starting branch from this module, you'll have everything you need. That's the Customer, Vehicle, RepairOrder, Repair, Part, and Quote entities. In each case, the entities have the properties that we defined on the diagram earlier, and I've added an integer Id property, which will become the primary key in the database table.

Data Access

For the most part, you can't build a modern business application without some kind of database. While this is not a course on database design, they play such a large part in business applications that I can't just not cover them at all. So let's create a database. I'll be using the latest version of Entity Framework Core, which is version 7 at the time I'm recording this. If you're interested in Entity Framework Core, then I'd recommend watching through the courses in Pluralsight's EF Core learning path. If not, then you should still be able to follow along just fine because I'll be sticking to pretty basic functionality that should be common to most object-relational mappers, or ORMs. You can think of an ORM as an object-oriented view of a database with intelligent objects representing database concepts like tables and views. The first step is to choose a database provider. For simple demonstrations like this, I like to use the SQLite provider since it's lightweight and only requires a single NuGet package to work with. I'll add the `Microsoft.EntityFrameworkCore.Sqlite` package, and then I'll also need the `Microsoft.EntityFrameworkCore.Tools` package in order to create migrations, so I'll add that while I'm here. Now, we can create our `DbContext` class, which represents the database as a whole in code. I'll call mine `RepairShopContext` and inherit it from `DbContext`. I'll add in `DbSet` properties, which represent the tables for each of our entities. Next, I'll need to configure the database to use SQLite, so I'll override the `OnConfiguring` method, and I'll call the `UseSqlite` method on `optionsBuilder` and give it a name for the database file. And that's all there is to it. In a real-world application, you might use a shared SQL Server or an Azure SQL Database, and you certainly would not hard code the connection string. But this is just a demo, and not a course on Entity Framework. So that's it for our database schema. A database without data is no fun, so we'll need to add some data so that we have something to work with. Entity Framework Core provides a fluent API for defining seed data, which will become part of our initial migration. I just need to override the `OnModelCreating` method and define some seed data for known repairs and parts. For each entity type, I call the `HasData` method on `modelBuilder` and

provide it with one or more entities of the appropriate type. I provide the IDs and any other fields that I want to be part of the seed data. The model is purposefully simplified for this demonstration. All engines use the same amount and type of oil and the same kind of filter. And tires are always replaced four at a time, etc. We just need some data to work with. I'll skip ahead here a little. I've filled in the seed data for some common repairs. There are routine maintenance tasks like oil changes and tire rotations, and then there are the actual repairs with categories to say which major system they relate to, like engine, transmission, etc. Recent versions of Entity Framework can work out many-to-many relationships on their own. And If I did nothing else, the resulting database would have a join table from the Repair table to the Part table. Since I want to seed that relationship though, I'll need to be a little more explicit about it. Repairs have many parts, and parts belong to many repairs. In a production system, the relationship between parts and repairs would probably specify a quantity as well, and there would be multiple types of brake pads and oil filters, all depending on the make and model of the vehicle. This is fine for our needs though. To actually create the database and fill it with the seed data, we'll need to create a migration, which is typically done using command line tools from the Package Manager Console. And that command is Add-Migration, and then we give it a name. And I like to be explicit about the Add-Migration command, and specify the project where the migration should go on the command line. This generates a class for us that describes the changes to be made to the database. Since this is the first migration, those changes include creating the schema and then inserting the seed data. As you can see, Entity Framework Correctly interpreted the nullability of the string fields, and the resulting migration will create nonnullable fields in the database. I'll apply the migrations using the Update-Database command, and we should be all set. As you can see, it created a database file for us, which has now appeared in the Solution Explorer. I'm going to use LINQPad to take a quick peek inside that database and check the results. You can use whatever tooling you're comfortable with, but it's not necessary for the rest of this module. I just want to show the results for those of you who don't normally use Entity Framework Core. The migration has created our database schema, and you can see that these string fields are all marked as NOT NULL. In older versions of Entity Framework, we would have had to apply attributes to our properties to make this happen, so the introduction of explicit nullability has paid off here. If I open the Repairs table, you can see that the seed data has been populated as well. So we're all set to start working on business logic.

Business Logic

Just as we thought carefully about where to put the data, we need to think about where to put the logic. There's no one best answer, only a best choice within the constraints of your project. I like to keep the business entities very simple. These kind of objects are sometimes called POCO objects for Plain Old CLR Object. It's a play on Martin Fowler's phrase Plain Old Java Object, but for .NET people, and it means a couple things. It means that the classes are very minimal, containing little more than simple properties, methods that operate on those properties, and connections to neighboring objects such as parents, children, or other directly-related entities. They are, however, completely unaware of the database and have to be loaded by external code. As such, they generally work well with ORMs like Entity Framework. If I decided to put the quote generation logic in a separate service rather than directly in the entities, something has to be responsible for loading the entities from the database, and since the entities can't do it themselves, this seemed like a logical choice. Let's create that QuoteService now. According to our design from earlier, the QuoteService takes a RepairOrder and returns an instance of the Quote class. In C#, that would look like this. We'll come back for the interface in a little while. Since the resulting quote will need to be saved to the database, we'll need an instance of the RepairShopContext, which we'll take in the constructor and store as an init only property. Since we have our own Context instance, we don't actually need the entire RepairOrder to be passed in. Instead, let's update the design to take the repair order's ID, and then we'll just load it ourselves. As for the logic, that's pretty simple. We load up the repairOrder, and assuming that the Id was valid, we create a new quote with a reference to the original RepairOrder. The PartTotal is the sum of all the prices of all the parts and all the repairs on the order. And labor is much the same. It's the sum of all the labor hours on all the repairs on the order, multiplied by some cost per hour. We could pull this value from the appSettings file at start up, using the options pattern from the last module, or we could pull it from a table in the database or from an environment variable, whatever makes the most sense. Let's add it as another parameter to the QuoteService constructor though so that QuoteService itself doesn't have to be concerned with loading it, which would violate the single-responsibility principle. The same goes for the quote duration too, but I think you get the idea. QuoteService's job is to generate quotes, not handle configuration issues. Once we've constructed the quote, we save it to the database and then return it to the caller. Object-oriented design is all about this division of labor, and deciding what goes where is always going to be a judgment call. I could have

put the quote construction logic inside the Quote class itself by taking a RepairOrder as a constructor parameter or by adding a factory method. I'm assuming that the generation of a real quote would be a lot more complicated, involving calls to a parts inventory system, a scheduling system, etc. These are all things that the Quote class, being a simple POCO class, wouldn't have direct access to, and that's why I've extracted those responsibilities to a service class. Every project is different, this won't fit every problem you ever run across, but it worked well for this demo. I think we're about done here, so let's extract that interface now. I don't want the Quote class to be completely passive in the this demo, so let's give it some of the work load too. Quotes are supposed to be able to generate a report that can be printed and given to the customer, so let's add that now by overriding the ToString method. Every object has a ToString method, and we can redefine it on any specific object type. Here, it's just a simple interpolated string, but it could be a chunk of XML or a JSON document to be fed to a report generator somewhere else. For the purposes of this demo, we'll just use a simple string though. Let's test it out. I don't want to go down a lot of rabbit holes, like creating a web API in order to ingest the repairOrder, so I'll be driving the whole thing from unit tests instead. Since we're touching the database, these are technically integration tests, but the idea is the same. I like to organize my tests according to the common Given-When-Then test structure. There are plenty of test libraries out there that make this possible, but I'm just using a couple of layered classes here to achieve the same effect. In the Given class, we construct the system under test and generate a RepairOrder to be used in the When class, when we pass its Id to the GenerateQuote method. Finally, in the Then methods, we write our various assertions. I want to test that the resulting quote has the expected part and labor totals and that the resulting report is generated correctly. If you want to run these tests, you'll want to pull the ending branch for this module. I've altered RepairShopContext to move the database to C:/Temp and made the services public for ease of testing. The tests pass, so we're done, right? No, we haven't covered even remotely half the topics from this course. Now, it would take a very long time to work every single topic we've covered into a single demonstration, so let's just pick one example pattern to cover some of the remaining topics. We put a placeholder in the class diagram for the WarrantyService earlier, but we didn't really define what it does at the time. Most vehicles will have multiple warranties in effect with almost everything covered by a bumper-to-bumper warranty for a certain length of time. A separate powertrain warranty might cover just the engine and transmission repairs for a longer period, and an emissions warranty might extend even further than that. Depending

on the type of the repair, the cost might be covered under multiple different warranties, other repairs might not be covered at all. This sounds like a job for a Strategy pattern. We're not going to build a complete commercial-grade warranty system here because that would be a huge undertaking, all we're really interested in is this division of labor, putting a separate class in charge of just one piece of the puzzle, in this case warranty coverage. First, let's design a base Warranty class. Each warranty has a duration and a maximum odometer reading. All implementations of the Warranty class will have an `IsCovered` method that takes an individual repair and returns a simple Boolean value, indicating whether or not that repair is covered. I've given the Duration and Odometer properties protected setters because it will be up to each implementation to decide those values. Also, it's worth noting here that the warranties are not entities. They're business logic classes, so they won't get entries in the data context. You certainly could build a data-driven version of the Warranty class, but I'm trying to show the Strategy pattern. There are multiple types of warranty, each with their own mysterious and inscrutable logic, so we'll inherit from the Warranty class to create several variations. So here we have our old friends inheritance and polymorphism at work. All warranties are interchangeable as far as the QuoteService is concerned. As long as they can answer the question is this repair covered, QuoteService doesn't care about their internal logic. Let's translate this diagram into code and build out the bumper-to-bumper Warranty class. The bumper-to-bumper warranty logic is very simple. It has a duration of 5 years and a maximum odometer reading of 50,000. It covers almost everything, except routine maintenance items like oil changes and flat tires. That's easy enough to implement. First, we'll check the duration and odometer limits. So if a vehicle is too old or has been driven too far, the warranty simply returns false; otherwise, it checks the repair against a list of repair categories that are covered. The powertrain and emissions warranties are similar, but have different durations, odometer limits, and cover fewer kinds of repairs. In the module on patterns, the different discount strategies each implemented a common interface. Here, the warranty implementations derive from a common base class, but the effect is the same, they're interchangeable. It may not match the Gang of Four Strategy pattern exactly, but it's still a viable implementation. Remember, you have some flexibility when it comes to implementing patterns. They're not some inviolable gospel that you must follow exactly. The WarrantyService class exposes a collection of warranty implementations, and when we submit a repair, it simply runs through the list and returns true if any of the implementations say that they cover the repair. If none of the loaded strategies cover the repair, then we return false. I'll update

the QuoteService class to take this information into account and then add new tests to cover the new behavior. If these calls to warrantyService were an expensive call to make, you'd probably want to turn this into a loop and cache the results of the call to IsCovered, but I'm using LINQ here to keep it small for this demo. I have both positive and negative tests for each warranty type. The radio would be covered under the bumper-to-bumper warranty until the odometer hits 50,000 or the 5 year warranty period is expired, an engine repair would be covered under the powertrain warranty up to 100,000 or 10 years, and replacing the muffler would be covered all the way up to 150,000 or 10 years. The QuoteService doesn't need to know any of that though, that's the WarrantyService's job, and the WarrantyService doesn't have to implement every different warranty because each warranty is just an implementation of the Strategy pattern.

Summary

In the end, object-oriented design is all about breaking up business problems into smaller, easily-digestible pieces. I know this was a long module, but let's take a moment to review what we built. We encapsulated the business information about customers, vehicles, repairs, quotes, parts, and labor into business entities. We abstracted logic into the classes holding the relevant information, such as the quote entity generating its own report. We inherited from a common warranty class to create several concrete implementations. And those subtypes were all substitutable for their base type, illustrating polymorphism. Although, I admit we didn't treat any one class as multiple different types here. Our entities were all focused on the representation of a single business concept, and our service classes stuck to their single responsibility instead of trying to do everything themselves. This extends to the use of Entity Framework as a general-purpose repository. QuoteService is still completely unaware of how entities are stored in the database. It only asks the context for the items it needs, and it doesn't concern itself with the rest of those details. Likewise, WarrantyService is blissfully unaware of how the individual warranties are implemented. It doesn't even know how many there are, so we can add more later if we want, and we won't have to open the WarrantyService class in order to add those implementations. We could discover and load them through reflection or even load them as plugins at runtime if we wanted. Any warranty implementation can be used, as they're all substitutable for their base class. We didn't exactly hit the Interface Segregation Principle in this example because it was so small, but we could have applied any number of different interfaces to the entities to

support functionality like validation if we wanted to. And both the QuoteService and WarrantyService were fed their dependencies from the outside. QuoteService never built its own instance of WarrantyService and only knows about it as an interface. It also didn't build its own DataContext class. And WarrantyService was fed the individual warranties from outside as well. We used modern C# features to declare our code's intent to its consumers by explicitly declaring the nullability of our entity properties. This informed Entity Framework Core, which was able to interpret that information to correctly build a database schema without us having to hold its hand or spoon feed it every detail. We used the Strategy pattern to create an extensible warranty service, and we could have made WarrantyService a singleton if we wanted to, rather than loading the warranty collection every time we want to use the service. While there isn't enough time to revisit every concept in the whole course in one demo, the opportunities are certainly there if you look for them. The Part class could implement value equality so that two instances of radio would be considered equal based on their ID or stock number properties. Instead of simply returning a string, the Quote class could have returned an immutable object representing the report. That object could have been implemented as a record. If we were operating against a remote database server, we could have loaded the connection string at startup using something like the options pattern. Just remember, you don't have to use every tool on every project. Be pragmatic and use what makes sense for the problems you're trying to solve. Hopefully, this course has helped you to see how these timeless pillars, principles, patterns, and practices can be realized using constructs in the modern C# language. I'm Mel Grubb, and I thank you for watching. And I hope to see you in the future, here on Pluralsight.